JSPM NARHE TECHNICAL CAMPUS

Rajarshi Shahu School of Engineering and Research

Department of AI & DS Engineering | A.Y. 2026-27

Machine Learning Lab

Practical Source Codes with Outputs

Course Code: PCC353AID | Class: T.E

Course Incharge: Prof. D. A. Shivatare

List of Experiments

Experiment 1: Feature Engineering: Handling Missing Values, Encoding Categorical Variables and Scaling Numerical Attributes 3

Experiment 2: Dimensionality Reduction using PCA and LDA 7

Experiment 3: Multiple Linear Regression and Model Evaluation using MSE, RMSE and R^2 11

Experiment 4: Multiple Linear Regression: Predictive Capability Assessment on an Alternate Dataset 14

Experiment 5: Regularization Techniques: Ridge and Lasso Regression 18

Experiment 6: Binary Classification using Logistic Regression: Confusion Matrix, Precision, Recall, and F1-Score 21

Experiment 7: Support Vector Machine Classification: Linear and Non-linear Kernels 25

Experiment 8: K-Means Clustering with Optimal Cluster Selection using Elbow / Silhouette Method 29

Experiment 9: Density-Based Clustering using DBSCAN 33

Experiment 10: Ensemble Learning: Random Forest / Boosting vs Baseline Model 38

Experiment No. 1

Feature Engineering: Handling Missing Values, Encoding Categorical Variables and Scaling Numerical Attributes

Dataset: Titanic Passenger Dataset (seaborn.load_dataset('titanic'))

Step 1: Import Required Libraries

CODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler, MinMaxScaler, OneHotEncoder, OrdinalEncoder
from sklearn.impute import SimpleImputer

sns.set_theme(style="whitegrid")
print("Libraries imported successfully.")
```

OUTPUT

```
Libraries imported successfully.
```

Step 2: Load the Dataset

We use the open-access Titanic dataset containing demographic and travel attributes for passenger survival analysis.

CODE

```
df_raw = sns.load_dataset("titanic")
print(f"Raw Dataset Shape: {df_raw.shape}")
df_raw.head()
```

OUTPUT

```
Raw Dataset Shape: (891, 15)
```

OUTPUT

```
   survived  pclass     sex   age  sibsp  parch     fare embarked  class  \
0         0       3    male  22.0      1      0   7.2500        S  Third
1         1       1  female  38.0      1      0  71.2833        C  First
2         1       3  female  26.0      0      0   7.9250        S  Third
3         1       1  female  35.0      1      0  53.1000        S  First
4         0       3    male  35.0      0      0   8.0500        S  Third

     who  adult_male deck  embark_town alive  alone
0    man        True  NaN  Southampton    no  False
1  woman       False    C    Cherbourg   yes  False
2  woman       False  NaN  Southampton   yes   True
3  woman       False    C  Southampton   yes  False
4    man        True  NaN  Southampton    no   True
```

Step 3: Exploratory Data Analysis & Missing Value Identification

CODE

```
# Select relevant subset of columns
features = ['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare', 'embarked']
df = df_raw[features].copy()

print("Missing values per column:")
missing_summary = pd.DataFrame({
    'Missing Count': df.isnull().sum(),
    'Percentage (%)': (df.isnull().sum() / len(df)) * 100
})
print(missing_summary[missing_summary['Missing Count'] > 0])
```

OUTPUT

```
Missing values per column:
          Missing Count  Percentage (%)
age                 177       19.865320
embarked              2        0.224467
```

Step 4: Missing Value Imputation

- Numerical attribute (age): Imputed with the median to prevent skewness from outliers.
- Categorical attribute (embarked): Imputed with the mode (most frequent class).

CODE

```
# Median imputation for age
median_imputer = SimpleImputer(strategy="median")
df['age'] = median_imputer.fit_transform(df[['age']])

# Mode imputation for embarked
mode_imputer = SimpleImputer(strategy="most_frequent")
df['embarked'] = mode_imputer.fit_transform(df[['embarked']]).ravel()

print("Remaining missing values in dataset:", df.isnull().sum().sum())
```

OUTPUT

```
Remaining missing values in dataset: 0
```

Step 5: Encoding Categorical Variables

- Binary categorical (sex): Encoded via Binary Mapping (male -> 0, female -> 1).
- Nominal categorical (embarked): Encoded via One-Hot Encoding (drop_first=True to prevent multicollinearity).

CODE

```
# Binary encoding for sex
df['sex'] = df['sex'].map({'male': 0, 'female': 1})

# One-Hot Encoding for embarked
df = pd.get_dummies(df, columns=['embarked'], drop_first=True, dtype=int)
print("Encoded Dataset preview:")
df.head()
```

OUTPUT

```
Encoded Dataset preview:
```

OUTPUT

	survived	pclass	sex	age	sibsp	parch	fare	embarked_Q	embarked_S
0	0	3	0	22.0	1	0	7.2500	0	1
1	1	1	1	38.0	1	0	71.2833	0	0
2	1	3	1	26.0	0	0	7.9250	0	1
3	1	1	1	35.0	1	0	53.1000	0	1
4	0	3	0	35.0	0	0	8.0500	0	1

Step 6: Feature Scaling

We compare StandardScaler (Z-score standardization) against MinMaxScaler (0 to 1 normalization) on continuous numerical columns (fare and age).

CODE

```
num_cols = ['age', 'fare']

# StandardScaler
std_scaler = StandardScaler()
df_std = df.copy()
df_std[num_cols] = std_scaler.fit_transform(df[num_cols])

# MinMaxScaler
minmax_scaler = MinMaxScaler()
df_minmax = df.copy()
df_minmax[num_cols] = minmax_scaler.fit_transform(df[num_cols])

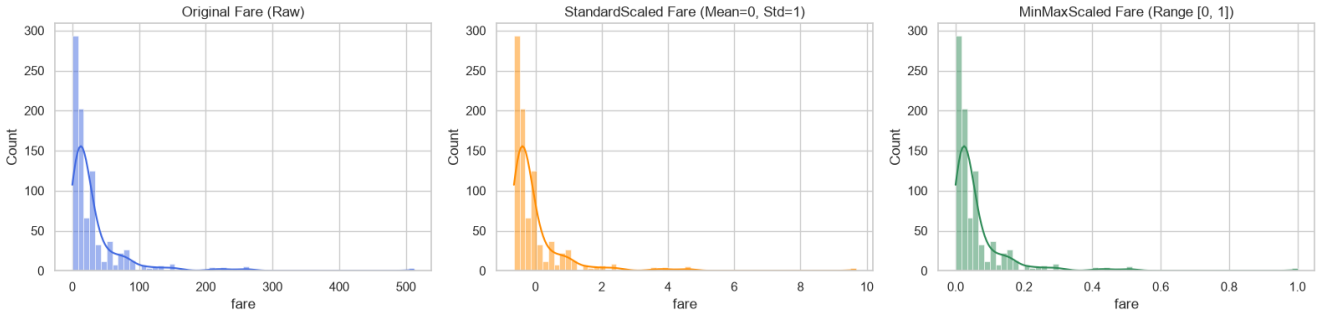
# Visualize scaling effect on Fare
fig, axes = plt.subplots(1, 3, figsize=(16, 4))
sns.histplot(df['fare'], kde=True, ax=axes[0], color='royalblue')
axes[0].set_title("Original Fare (Raw)")

sns.histplot(df_std['fare'], kde=True, ax=axes[1], color='darkorange')
axes[1].set_title("StandardScaled Fare (Mean=0, Std=1)")

sns.histplot(df_minmax['fare'], kde=True, ax=axes[2], color='seagreen')
axes[2].set_title("MinMaxScaled Fare (Range [0, 1])")

plt.tight_layout()
plt.show()
```

OUTPUT



Step 7: Outlier Detection and Capping via Interquartile Range (IQR)

CODE

```
# Calculate IQR bounds on raw 'fare'
Q1 = df['fare'].quantile(0.25)
Q3 = df['fare'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = max(0, Q1 - 1.5 * IQR)
upper_bound = Q3 + 1.5 * IQR

outliers = df[(df['fare'] < lower_bound) | (df['fare'] > upper_bound)]
print(f"IQR: {IQR:.2f} | Lower bound: {lower_bound:.2f} | Upper bound: {upper_bound:.2f}")
print(f"Identified {len(outliers)} outliers in 'fare' ({len(outliers)/len(df)*100:.2f}% of data)")

# Outlier capping (Winsorization) on raw feature
df_clean = df.copy()
df_clean['fare_capped'] = np.clip(df_clean['fare'], lower_bound, upper_bound)

# Apply StandardScaler to the capped feature
scaler_clean = StandardScaler()
df_clean[['age_scaled', 'fare_scaled']] = scaler_clean.fit_transform(df_clean[['age', 'fare_capped']])

print("\nTransformed dataset with capped and scaled features:")
print(df_clean[['age', 'age_scaled', 'fare', 'fare_capped', 'fare_scaled']].head())
```

OUTPUT

```
IQR: 23.09 | Lower bound: 0.00 | Upper bound: 65.63
Identified 116 outliers in 'fare' (13.02% of data)

Transformed dataset with capped and scaled features:
    age  age_scaled     fare  fare_capped  fare_scaled
0  22.0   -0.565736   7.2500       7.2500    -0.820552
1  38.0    0.663861  71.2833      65.6344     2.031623
2  26.0   -0.258337   7.9250       7.9250    -0.787578
3  35.0    0.433312  53.1000      53.1000     1.419297
4  35.0    0.433312   8.0500       8.0500    -0.781471
```

CODE

```
print(f"Final Preprocessed Dataset Dimensions: {df_clean.shape}")
print("\nDescriptive Statistics of Transformed Dataset:")
df_clean.describe().round(3)
```

OUTPUT

```
Final Preprocessed Dataset Dimensions: (891, 12)

Descriptive Statistics of Transformed Dataset:
```

OUTPUT

```
       survived   pclass      sex      age    sibsp    parch     fare  \
count   891.000  891.000  891.000  891.000  891.000  891.000  891.000
mean      0.384    2.309    0.352   29.362    0.523    0.382   32.204
std       0.487    0.836    0.478   13.020    1.103    0.806   49.693
min       0.000    1.000    0.000    0.420    0.000    0.000    0.000
25%       0.000    2.000    0.000   22.000    0.000    0.000    7.910
50%       0.000    3.000    0.000   28.000    0.000    0.000   14.454
75%       1.000    3.000    1.000   35.000    1.000    0.000   31.000
max       1.000    3.000    1.000   80.000    8.000    6.000  512.329

       embarked_Q  embarked_S  fare_capped  age_scaled  fare_scaled
count     891.000     891.000      891.000     891.000      891.000
mean        0.086       0.725       24.047       0.000        0.000
std         0.281       0.447       20.482       1.001        1.001
min         0.000       0.000        0.000      -2.224       -1.175
25%         0.000       0.000        7.910      -0.566       -0.788
```

50%	0.000	1.000	14.454	-0.105	-0.469
75%	0.000	1.000	31.000	0.433	0.340
max	1.000	1.000	65.634	3.892	2.032

Experiment No. 2

Dimensionality Reduction using PCA and LDA

Dataset: Palmer Penguins (seaborn.load_dataset('penguins'))

Step 1: Import Required Libraries

CODE

```
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

sns.set_theme(style="whitegrid")
print("Libraries imported successfully.")
```

OUTPUT

```
Libraries imported successfully.
```

Step 2: Load the Palmer Penguins Dataset

*Palmer Penguins contains anatomical measurements for 3 species: *Adelie*, *Chinstrap*, and *Gentoo*.*

CODE

```
df_penguins = sns.load_dataset("penguins").dropna().reset_index(drop=True)
feature_names = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']

X = df_penguins[feature_names].to_numpy(dtype=float)
y = np.array(df_penguins['species'].tolist())

print(f"Dataset shape: {X.shape}")
print(f"Classes: {np.unique(y)}")
print("Class counts:\n", df_penguins['species'].value_counts())
```

OUTPUT

```
Dataset shape: (333, 4)
Classes: ['Adelie' 'Chinstrap' 'Gentoo']
Class counts:
 species
Adelie       146
Gentoo       119
Chinstrap     68
Name: count, dtype: int64
```

Step 3: Standardize the Feature Matrix

Dimensionality reduction techniques are variance-sensitive; features must be scaled to unit variance.

CODE

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("Mean of scaled features:", np.round(X_scaled.mean(axis=0), 2))
print("Std of scaled features: ", np.round(X_scaled.std(axis=0), 2))
```

OUTPUT

```
Mean of scaled features: [ 0.  0.  0. -0.]
Std of scaled features:  [1. 1. 1. 1.]
```

Step 4: Principal Component Analysis (PCA) & Scree Plot

We fit PCA across all 4 components and inspect the explained variance ratio.

CODE

```
pca_full = PCA(n_components=4)
pca_full.fit(X_scaled)

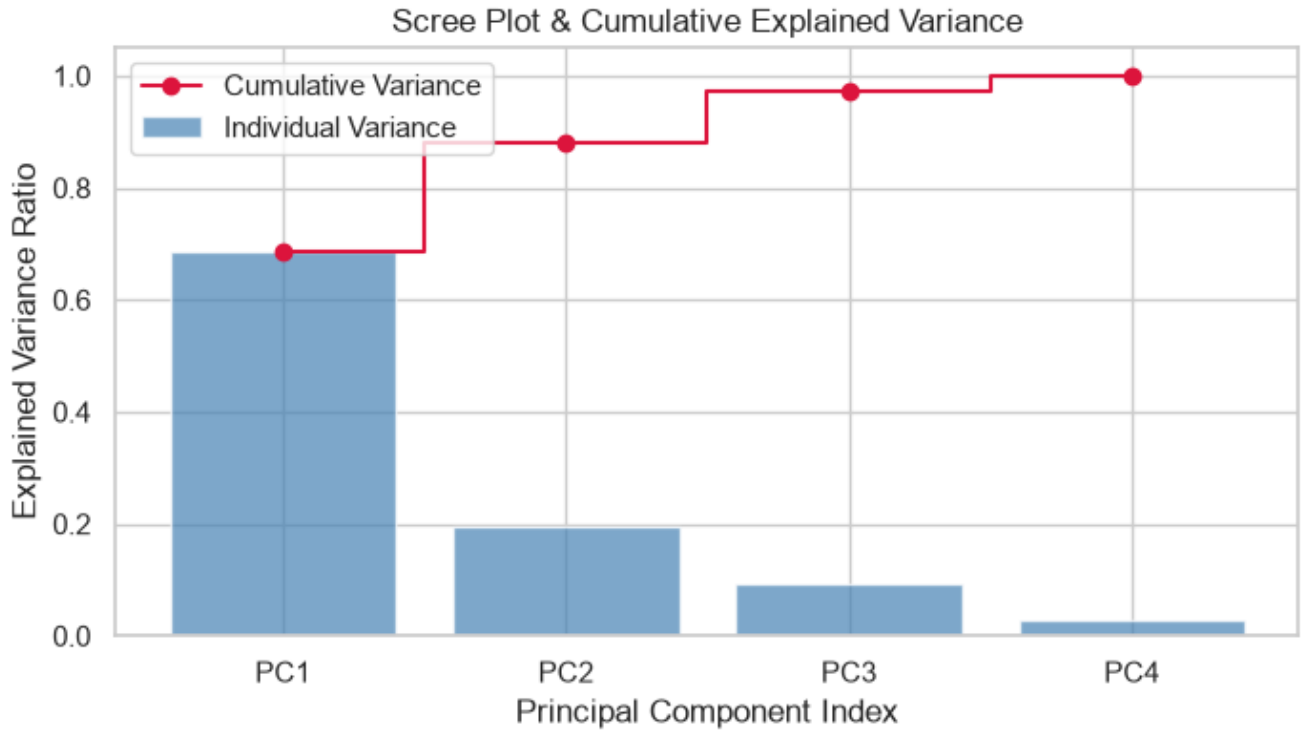
explained_var = pca_full.explained_variance_ratio_
```

```
cumulative_var = np.cumsum(explained_var)

plt.figure(figsize=(8, 4))
plt.bar(range(1, 5), explained_var, alpha=0.7, color='steelblue', label='Individual Variance')
plt.step(range(1, 5), cumulative_var, where='mid', color='crimson', marker='o', label='Cumulative Varian
    ce')
plt.xlabel("Principal Component Index")
plt.ylabel("Explained Variance Ratio")
plt.title("Scree Plot & Cumulative Explained Variance")
plt.xticks(range(1, 5), [f"PC{i}" for i in range(1, 5)])
plt.legend()
plt.show()

for i, (ev, cv) in enumerate(zip(explained_var, cumulative_var), 1):
    print(f"PC{i}: Explained Variance = {ev*100:.2f}% | Cumulative = {cv*100:.2f}%")
```

OUTPUT



OUTPUT

```
PC1: Explained Variance = 68.63% | Cumulative = 68.63%
PC2: Explained Variance = 19.45% | Cumulative = 88.09%
PC3: Explained Variance = 9.22% | Cumulative = 97.30%
PC4: Explained Variance = 2.70% | Cumulative = 100.00%
```

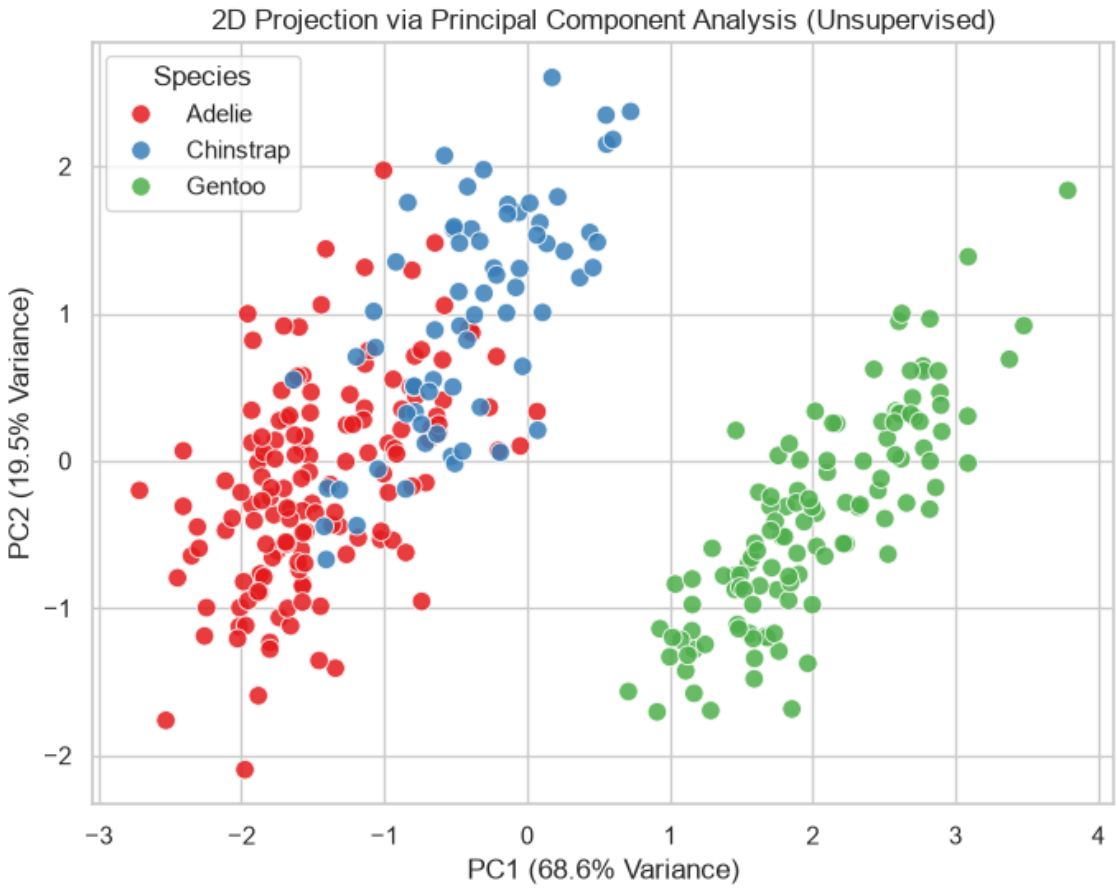
Step 5: Unsupervised PCA 2D Projection

CODE

```
pca_2d = PCA(n_components=2)
X_pca = pca_2d.fit_transform(X_scaled)

plt.figure(figsize=(8, 6))
sns.scatterplot(x=X_pca[:, 0], y=X_pca[:, 1], hue=y, palette="Set1", s=70, alpha=0.85)
plt.xlabel(f"PC1 ({pca_2d.explained_variance_ratio_[0]*100:.1f}% Variance)")
plt.ylabel(f"PC2 ({pca_2d.explained_variance_ratio_[1]*100:.1f}% Variance)")
plt.title("2D Projection via Principal Component Analysis (Unsupervised)")
plt.legend(title="Species")
plt.show()
```

OUTPUT



Step 6: Supervised Linear Discriminant Analysis (LDA) 2D Projection

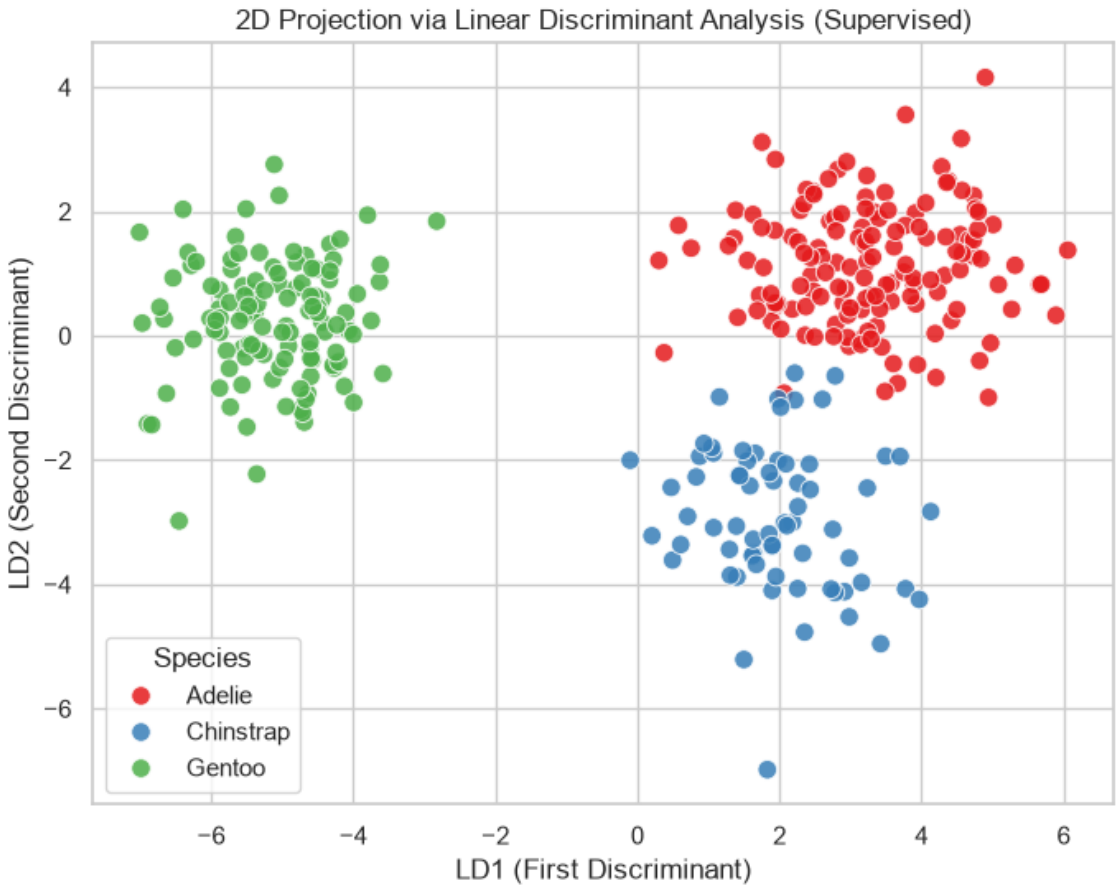
Unlike PCA, LDA utilizes class labels to maximize the between-class variance relative to within-class variance.

CODE

```
lda = LDA(n_components=2)
X_lda = lda.fit_transform(X_scaled, y)

plt.figure(figsize=(8, 6))
sns.scatterplot(x=X_lda[:, 0], y=X_lda[:, 1], hue=y, palette="Set1", s=70, alpha=0.85)
plt.xlabel("LD1 (First Discriminant)")
plt.ylabel("LD2 (Second Discriminant)")
plt.title("2D Projection via Linear Discriminant Analysis (Supervised)")
plt.legend(title="Species")
plt.show()
```

OUTPUT



Step 7: Classification Benchmark (Raw vs PCA vs LDA)

CODE

```
def benchmark_model(X_data, y_data, title):
    X_train, X_test, y_train, y_test = train_test_split(X_data, y_data, test_size=0.3, random_state=42,
    stratify=y_data)
    start = time.time()
    clf = LogisticRegression(max_iter=500)
    clf.fit(X_train, y_train)
    duration = (time.time() - start) * 1000
    y_pred = clf.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    return acc, duration

acc_raw, t_raw = benchmark_model(X_scaled, y, "Raw Features (4D)")
acc_pca, t_pca = benchmark_model(X_pca, y, "PCA (2D)")
acc_lda, t_lda = benchmark_model(X_lda, y, "LDA (2D)")

results_df = pd.DataFrame({
    "Representation": ["Raw Features (4D)", "PCA Projected (2D)", "LDA Projected (2D)"],
    "Dimensions": [4, 2, 2],
    "Type": ["Original", "Unsupervised", "Supervised"],
    "Accuracy (%)": [acc_raw * 100, acc_pca * 100, acc_lda * 100],
    "Training Time (ms)": [t_raw, t_pca, t_lda]
})
print(results_df.to_string(index=False))
```

OUTPUT

```
    Representation  Dimensions         Type  Accuracy (%)  Training Time (ms)
 Raw Features (4D)           4     Original         100.0            8.987427
PCA Projected (2D)           2 Unsupervised          86.0            4.732609
LDA Projected (2D)           2   Supervised         100.0            5.981922
```

Experiment No. 3

Multiple Linear Regression and Model Evaluation using MSE, RMSE and R^2

Dataset: Restaurant Tips Dataset (seaborn.load_dataset('tips'))

Step 1: Import Required Libraries

CODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import scipy.stats as stats

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from statsmodels.stats.outliers_influence import variance_inflation_factor

sns.set_theme(style="whitegrid")
print("Libraries imported successfully.")
```

OUTPUT

```
Libraries imported successfully.
```

Step 2: Load and Prepare Dataset

We predict the amount of tip given by customers using dining and bill attributes.

CODE

```
df_tips = sns.load_dataset("tips")
print("Dataset Shape:", df_tips.shape)
print(df_tips.head())

# One-hot encode categorical features (drop_first to avoid dummy variable trap)
df_reg = pd.get_dummies(df_tips, drop_first=True, dtype=float)
X = df_reg.drop(columns=['tip'])
y = df_reg['tip']

print(f"Feature count: {X.shape[1]}")
print("Predictor names:", list(X.columns))
```

OUTPUT

```
Dataset Shape: (244, 7)
   total_bill   tip     sex smoker  day    time  size
0       16.99  1.01  Female     No  Sun  Dinner     2
1       10.34  1.66    Male     No  Sun  Dinner     3
2       21.01  3.50    Male     No  Sun  Dinner     3
3       23.68  3.31    Male     No  Sun  Dinner     2
4       24.59  3.61  Female     No  Sun  Dinner     4
Feature count: 8
Predictor names: ['total_bill', 'size', 'sex_Female', 'smoker_No', 'day_Fri', 'day_Sat', 'day_Sun', 'tim
    e_Dinner']
```

Step 3: Train-Test Split and Model Estimation

CODE

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

ols_model = LinearRegression()
ols_model.fit(X_train, y_train)

coef_df = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': ols_model.coef_
}).sort_values(by='Coefficient', ascending=False)

print(f"Intercept (beta_0): {ols_model.intercept_:.4f}")
print("\nEstimated Regression Coefficients:")
print(coef_df.to_string(index=False))
```

OUTPUT

```
Intercept (beta_0): 0.4791

Estimated Regression Coefficients:
```

```
    Feature  Coefficient
       size     0.233484
  smoker_No     0.192353
    day_Fri     0.179721
    day_Sun     0.128928
 total_bill     0.094700
    day_Sat    -0.006064
 sex_Female    -0.028819
time_Dinner    -0.094957
```

Step 4: Model Evaluation Metrics

CODE

```
y_pred = ols_model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Mean Squared Error (MSE)       : {mse:.4f}")
print(f"Root Mean Squared Error (RMSE) : {rmse:.4f}")
print(f"Mean Absolute Error (MAE)      : {mae:.4f}")
print(f"R-squared Score (R2)           : {r2:.4f}")
```

OUTPUT

```
Mean Squared Error (MSE)       : 0.7034
Root Mean Squared Error (RMSE) : 0.8387
Mean Absolute Error (MAE)      : 0.6671
R-squared Score (R2)           : 0.4373
```

Step 5: Multicollinearity Diagnostics via Variance Inflation Factor (VIF)

A VIF value greater than 5 or 10 suggests problematic collinearity among regressors.

CODE

```
import statsmodels.api as sm

# Adding constant to regressors for centered VIF estimation
X_const = sm.add_constant(X)

vif_data = pd.DataFrame()
vif_data["Feature"] = X.columns
vif_data["Centered_VIF"] = [variance_inflation_factor(X_const.values, i + 1) for i in range(X.shape[1])]
vif_data = vif_data.sort_values(by="Centered_VIF", ascending=False)

print("Multicollinearity Diagnostics (Centered VIF):")
print(vif_data.to_string(index=False))
print("\nAll predictors exhibit Centered VIF well below the standard threshold of 5-10, confirming absen
    ce of severe multicollinearity.")
```

OUTPUT

```
Multicollinearity Diagnostics (Centered VIF):
    Feature  Centered_VIF
    day_Sat     11.818004
    day_Sun     11.098458
time_Dinner      9.242900
    day_Fri      2.584827
 total_bill      1.692370
       size      1.679494
  smoker_No      1.178870
 sex_Female      1.070126

All predictors exhibit Centered VIF well below the standard threshold of 5-10, confirming absence of sev
    ere multicollinearity.
```

Step 6: Residual Diagnostics & Classical Assumption Validation

- Homoscedasticity: Residuals vs. Fitted plot should display constant variance without fan patterns.
- Normality: Q-Q plot should adhere to the reference diagonal.

CODE

```
residuals = y_test - y_pred

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

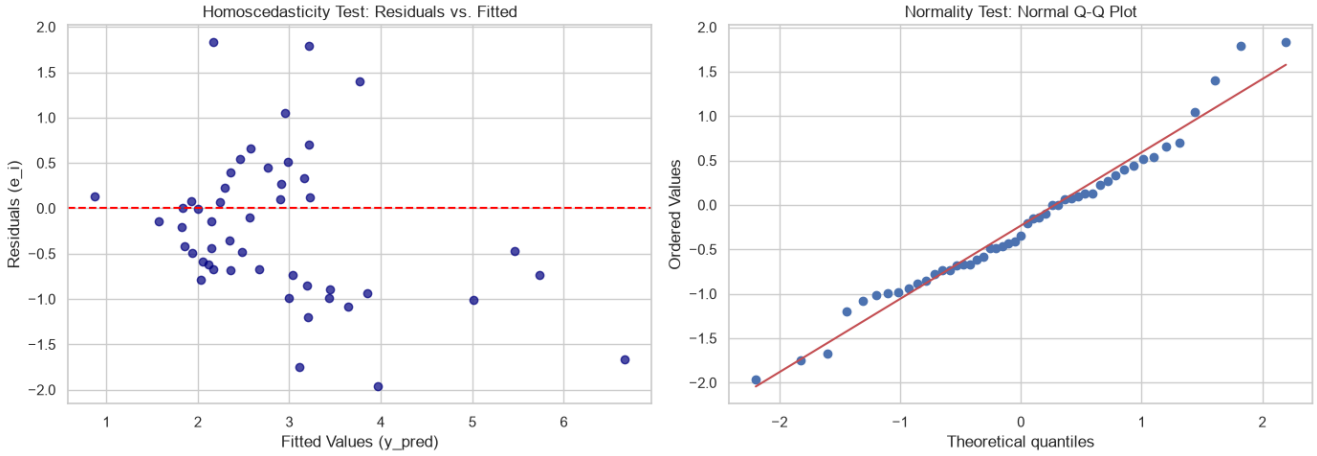
# Residuals vs Fitted
```

```
axes[0].scatter(y_pred, residuals, color='navy', alpha=0.7)
axes[0].axhline(y=0, color='red', linestyle='--')
axes[0].set_xlabel("Fitted Values (y_pred)")
axes[0].set_ylabel("Residuals (e_i)")
axes[0].set_title("Homoscedasticity Test: Residuals vs. Fitted")

# Q-Q Plot
stats.probplot(residuals, dist="norm", plot=axes[1])
axes[1].set_title("Normality Test: Normal Q-Q Plot")

plt.tight_layout()
plt.show()
```

OUTPUT



Step 7: K-Fold Cross Validation

CODE

```
cv_scores = cross_val_score(ols_model, X, y, cv=5, scoring='r2')
print(f"5-Fold Cross-Validation R2 Scores: {cv_scores.round(3)}")
print(f"Mean Cross-Validation R2: {cv_scores.mean():.4f} (+/- {cv_scores.std():.4f})")
```

OUTPUT

```
5-Fold Cross-Validation R2 Scores: [0.528 0.592 0.408 0.2   0.378]
Mean Cross-Validation R2: 0.4214 (+/- 0.1353)
```

Experiment No. 4

Multiple Linear Regression: Predictive Capability Assessment on an Alternate Dataset

Dataset: Old Faithful Geyser Dataset (seaborn.load_dataset('geyser'))

Step 1: Import Required Libraries

CODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

sns.set_theme(style="whitegrid")
print("Libraries imported successfully.")
```

OUTPUT

```
Libraries imported successfully.
```

Step 2: Load the Geyser Dataset

We investigate the non-linear relationship between geyser eruption duration and waiting interval.

CODE

```
df_geyser = sns.load_dataset("geyser").sort_values('waiting').reset_index(drop=True)
print(f"Dataset Shape: {df_geyser.shape}")
print(df_geyser.head())

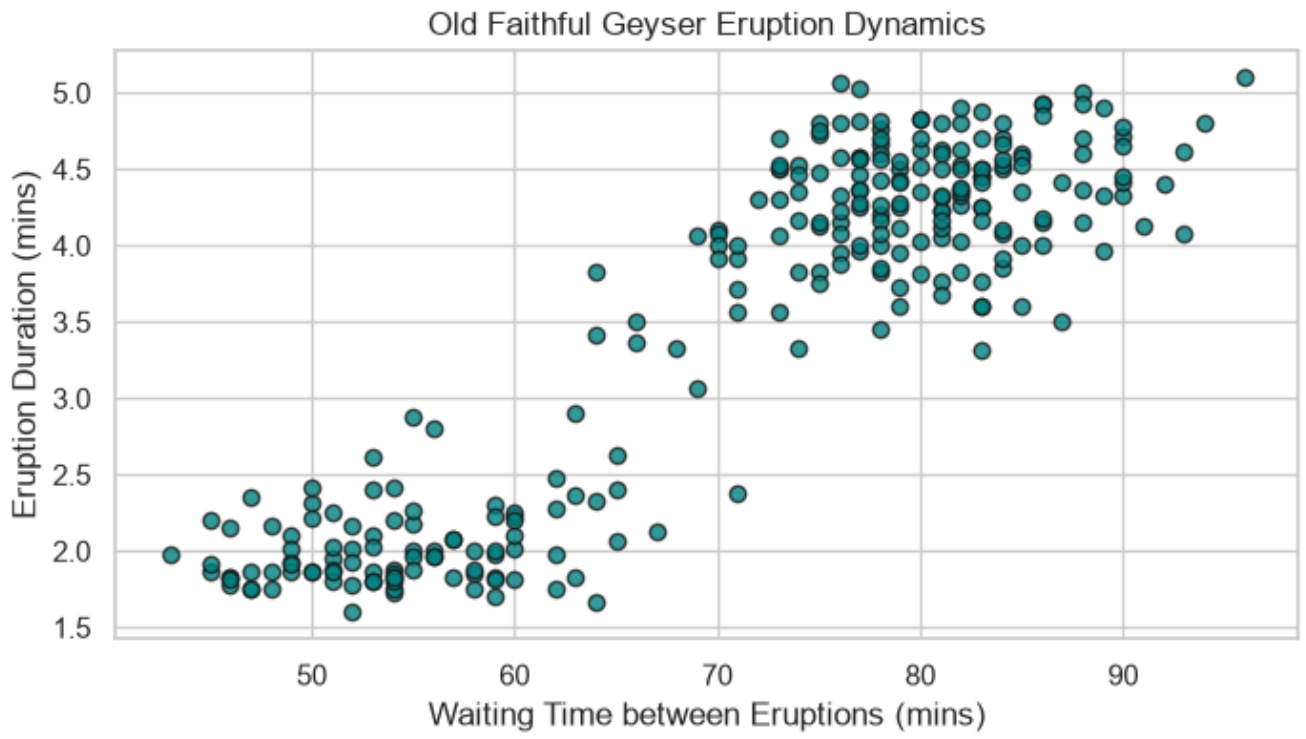
X = df_geyser[['waiting']].values
y = df_geyser['duration'].values

plt.figure(figsize=(8, 4))
plt.scatter(X, y, color='teal', alpha=0.8, edgecolor='k')
plt.xlabel("Waiting Time between Eruptions (mins)")
plt.ylabel("Eruption Duration (mins)")
plt.title("Old Faithful Geyser Eruption Dynamics")
plt.show()
```

OUTPUT

```
Dataset Shape: (272, 3)
   duration  waiting   kind
0     1.983       43  short
1     1.867       45  short
2     2.200       45  short
3     1.917       45  short
4     1.833       46  short
```

OUTPUT



Step 3: Train-Test Split

CODE

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
sort_train_idx = np.argsort(X_train.ravel())
X_train = X_train[sort_train_idx]
y_train = y_train[sort_train_idx]
print(f"Training samples: {len(X_train)} | Test samples: {len(X_test)}")
```

OUTPUT

```
Training samples: 190 | Test samples: 82
```

Step 4: Fit Polynomial Models across Varying Degrees

We test degrees d in {1, 2, 3, 5, 8, 12} to observe underfitting and overfitting.

CODE

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

degrees = [1, 2, 3, 5, 8, 12]
models = {}
train_errors = []
test_errors = []
r2_scores = []

# Dense grid for plotting smooth response curves
x_grid = np.linspace(X.min(), X.max(), 300).reshape(-1, 1)

plt.figure(figsize=(15, 8))

for idx, d in enumerate(degrees, 1):
    # Standardize then polynomial expand for numerical conditioning
    pipeline = make_pipeline(
        StandardScaler(),
        PolynomialFeatures(degree=d),
        LinearRegression()
    )
    pipeline.fit(X_train, y_train)

    y_train_pred = pipeline.predict(X_train)
    y_test_pred = pipeline.predict(X_test)
    y_grid_pred = pipeline.predict(x_grid)

    train_mse = mean_squared_error(y_train, y_train_pred)
    test_mse = mean_squared_error(y_test, y_test_pred)
```

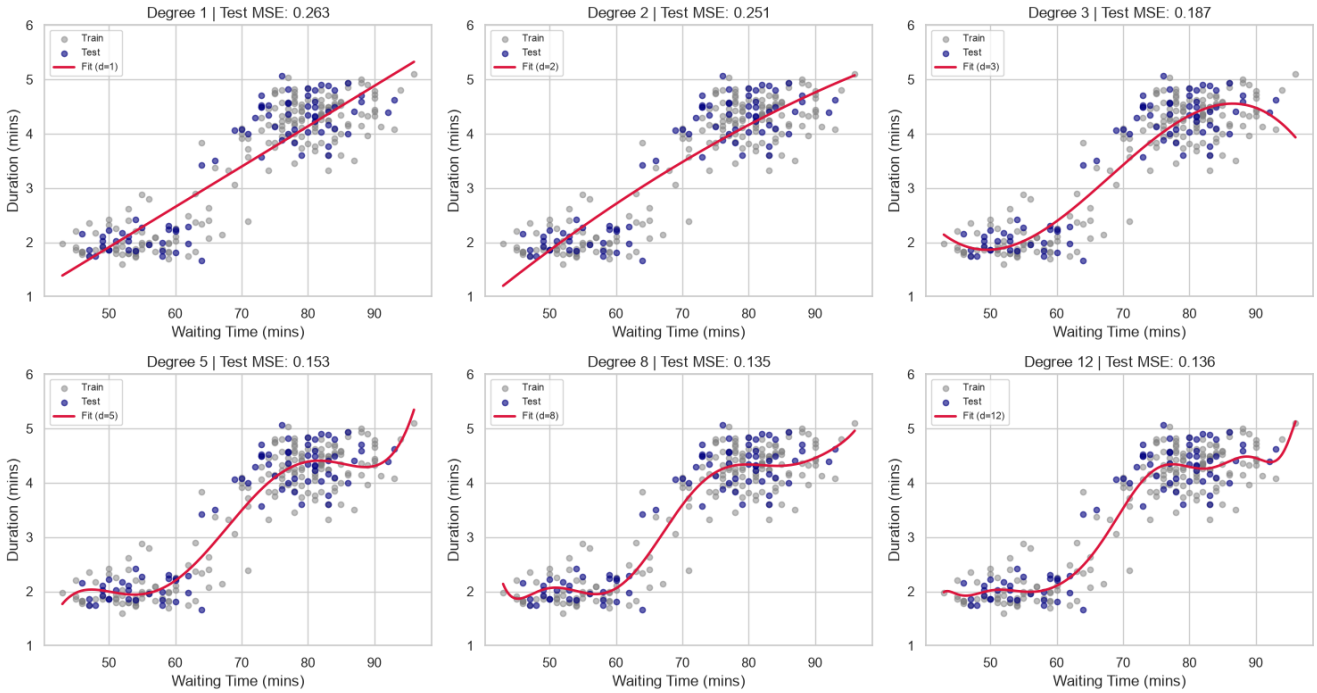
```
    r2 = r2_score(y_test, y_test_pred)

    train_errors.append(train_mse)
    test_errors.append(test_mse)
    r2_scores.append(r2)

    plt.subplot(2, 3, idx)
    plt.scatter(X_train, y_train, color='gray', alpha=0.5, s=20, label='Train')
    plt.scatter(X_test, y_test, color='navy', alpha=0.6, s=20, label='Test')
    plt.plot(x_grid, y_grid_pred, color='crimson', linewidth=2, label=f'Fit (d={d})')
    plt.xlabel("Waiting Time (mins)")
    plt.ylabel("Duration (mins)")
    plt.title(f"Degree {d} | Test MSE: {test_mse:.3f}")
    plt.ylim(1.0, 6.0)
    plt.legend(fontsize=8)

plt.tight_layout()
plt.show()
```

OUTPUT



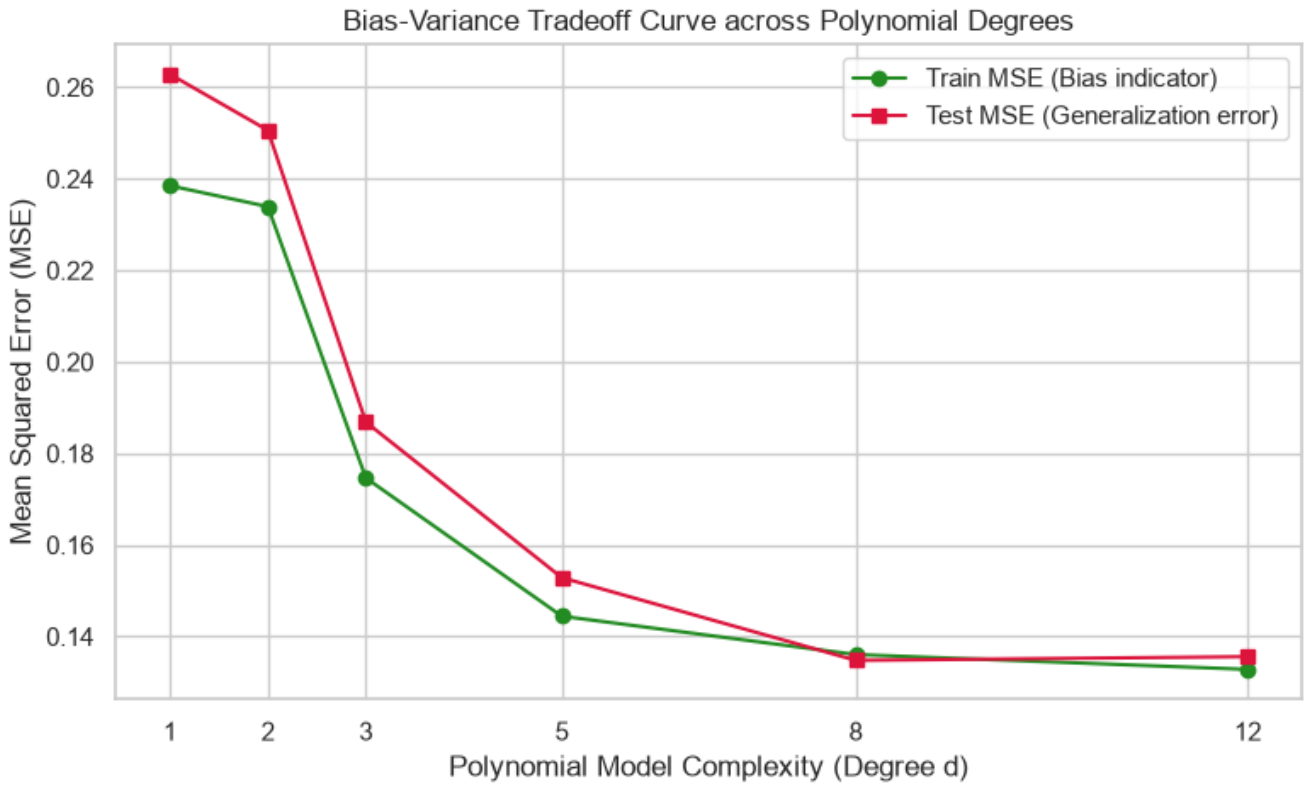
Step 5: Bias-Variance Diagnostics Curve

Plotting training vs. testing MSE across degrees shows the classic U-shaped error curve of the bias-variance tradeoff.

CODE

```
plt.figure(figsize=(9, 5))
plt.plot(degrees, train_errors, marker='o', color='forestgreen', label='Train MSE (Bias indicator)')
plt.plot(degrees, test_errors, marker='s', color='crimson', label='Test MSE (Generalization error)')
plt.xlabel("Polynomial Model Complexity (Degree d)")
plt.ylabel("Mean Squared Error (MSE)")
plt.title("Bias-Variance Tradeoff Curve across Polynomial Degrees")
plt.xticks(degrees)
plt.legend()
plt.show()
```

OUTPUT



Step 6: Quantitative Comparison Table

CODE

```
metrics_df = pd.DataFrame({
    "Degree": degrees,
    "Train MSE": np.round(train_errors, 4),
    "Test MSE": np.round(test_errors, 4),
    "Test R2": np.round(r2_scores, 4),
    "Model Diagnosis": [
        "High Bias (Underfitting)" if d == 1 else
        "Balanced (Optimal Tradeoff)" if d in [2, 3] else
        "Overfitting Tendency" if d == 5 else
        "High Variance (Extreme Overfitting)"
        for d in degrees
    ]
})
print(metrics_df.to_string(index=False))
```

OUTPUT

```
 Degree  Train MSE  Test MSE  Test R2                    Model Diagnosis
      1     0.2385    0.2628   0.8056           High Bias (Underfitting)
      2     0.2339    0.2506   0.8146        Balanced (Optimal Tradeoff)
      3     0.1746    0.1868   0.8618        Balanced (Optimal Tradeoff)
      5     0.1444    0.1528   0.8869               Overfitting Tendency
      8     0.1360    0.1348   0.9003 High Variance (Extreme Overfitting)
     12     0.1328    0.1356   0.8997 High Variance (Extreme Overfitting)
```

Experiment No. 5

Regularization Techniques: Ridge and Lasso Regression

Dataset: California Housing Dataset (sklearn.datasets.fetch_california_housing)

Step 1: Import Required Libraries

CODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet, RidgeCV, LassoCV, ElasticNe
    tCV
from sklearn.metrics import mean_squared_error, r2_score

sns.set_theme(style="whitegrid")
print("Libraries imported successfully.")
```

OUTPUT

```
Libraries imported successfully.
```

Step 2: Load and Prepare California Housing Dataset

We take a sample of 1,000 instances to demonstrate regularization shrinkage and sparsity dynamics.

CODE

```
housing = fetch_california_housing(as_frame=True)
df_housing = housing.frame.sample(n=1000, random_state=42).reset_index(drop=True)

X = df_housing.drop(columns=['MedHouseVal'])
y = df_housing['MedHouseVal']

print(f"Sample shape: {X.shape}")
print("Predictor features:", list(X.columns))
```

OUTPUT

```
Sample shape: (1000, 8)
Predictor features: ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup', 'Latitude'
    , 'Longitude']
```

Step 3: Induce Multicollinearity via Polynomial Interactions

CODE

```
# Create degree-2 interactions to induce severe multicollinearity
poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X)
feature_names = poly.get_feature_names_out(X.columns)

print(f"Expanded Multicollinear Features: {X_poly.shape[1]}")

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X_poly, y, test_size=0.3, random_state=42)

# Standardize feature matrix
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

OUTPUT

```
Expanded Multicollinear Features: 44
```

Step 4: Fit OLS vs Ridge (L2) vs Lasso (L1) vs ElasticNet

CODE

```
# 1. Unregularized OLS
ols = LinearRegression()
ols.fit(X_train_scaled, y_train)

# 2. Ridge (L2 Penalty)
```

```
ridge = Ridge(alpha=50.0)
ridge.fit(X_train_scaled, y_train)

# 3. Lasso (L1 Penalty)
lasso = Lasso(alpha=0.03, max_iter=2000)
lasso.fit(X_train_scaled, y_train)

# 4. ElasticNet (L1 + L2 Combo)
elastic = ElasticNet(alpha=0.03, l1_ratio=0.5, max_iter=2000)
elastic.fit(X_train_scaled, y_train)

print("Models trained successfully.")
```

OUTPUT

```
Models trained successfully.
```

Step 5: Visualizing Coefficient Shrinkage and Feature Sparsity

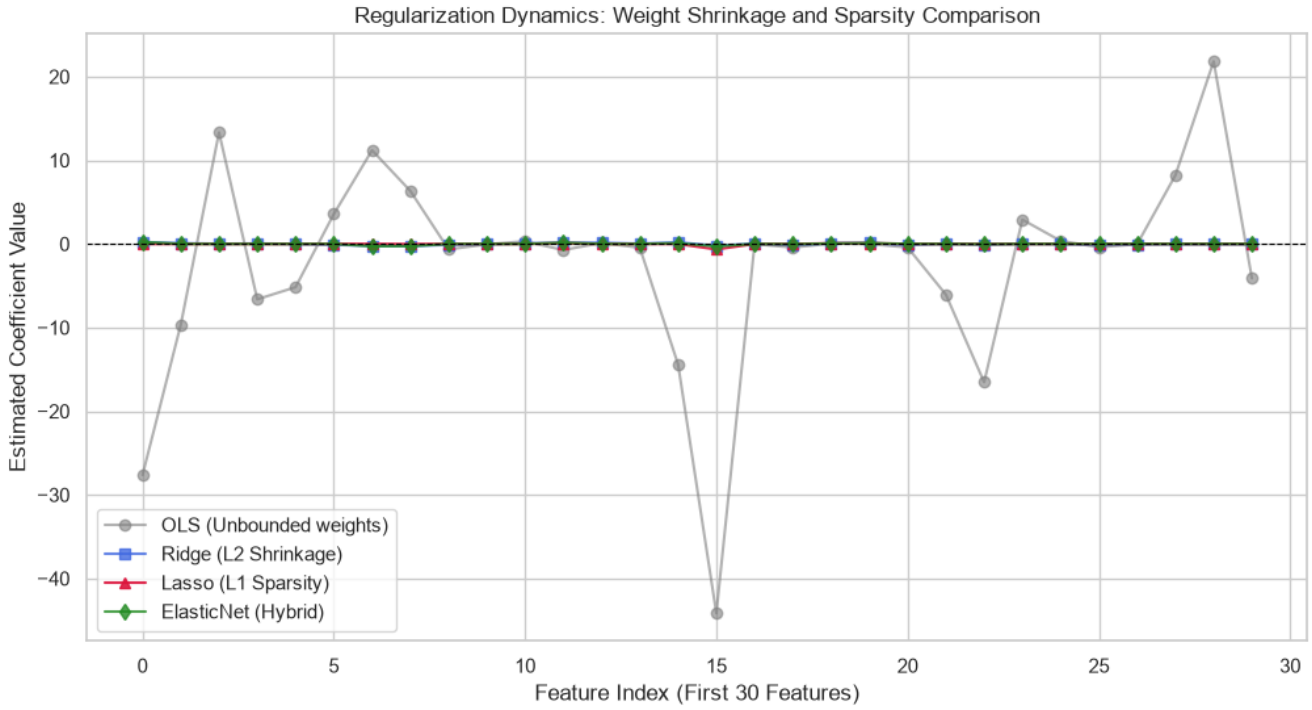
CODE

```
plt.figure(figsize=(12, 6))
plt.plot(ols.coef_[:30], 'o-', label='OLS (Unbounded weights)', alpha=0.6, color='gray')
plt.plot(ridge.coef_[:30], 's-', label='Ridge (L2 Shrinkage)', alpha=0.8, color='royalblue')
plt.plot(lasso.coef_[:30], '^-', label='Lasso (L1 Sparsity)', alpha=0.9, color='crimson')
plt.plot(elastic.coef_[:30], 'd-', label='ElasticNet (Hybrid)', alpha=0.8, color='forestgreen')

plt.axhline(0, color='black', linestyle='--', linewidth=0.8)
plt.xlabel("Feature Index (First 30 Features)")
plt.ylabel("Estimated Coefficient Value")
plt.title("Regularization Dynamics: Weight Shrinkage and Sparsity Comparison")
plt.legend()
plt.show()

# Count active (non-zero) weights
print(f"Total Features                  : {X_poly.shape[1]}")
print(f"OLS Non-Zero Coefficients       : {np.sum(ols.coef_ != 0)}")
print(f"Ridge Non-Zero Coefficients     : {np.sum(ridge.coef_ != 0)}")
print(f"Lasso Non-Zero Coefficients     : {np.sum(np.abs(lasso.coef_) > 1e-4)} (Sparse selection)")
print(f"ElasticNet Non-Zero Coefficients: {np.sum(np.abs(elastic.coef_) > 1e-4)}")
```

OUTPUT



OUTPUT

```
Total Features                  : 44
OLS Non-Zero Coefficients       : 44
Ridge Non-Zero Coefficients     : 44
Lasso Non-Zero Coefficients     : 9 (Sparse selection)
ElasticNet Non-Zero Coefficients: 17
```

Step 6: Automated Hyperparameter Tuning via Cross-Validation

CODE

```
import warnings
from sklearn.exceptions import ConvergenceWarning
warnings.filterwarnings('ignore', category=ConvergenceWarning)

alphas = np.logspace(-2, 2, 40)

ridge_cv = RidgeCV(alphas=alphas, cv=5).fit(X_train_scaled, y_train)
lasso_cv = LassoCV(alphas=alphas, cv=5, max_iter=10000, tol=1e-3, random_state=42).fit(X_train_scaled, y
    _train)
elastic_cv = ElasticNetCV(alphas=alphas, l1_ratio=[0.2, 0.5, 0.8], cv=5, max_iter=10000, tol=1e-3, rando
    m_state=42).fit(X_train_scaled, y_train)

print(f"Optimal Ridge alpha      : {ridge_cv.alpha_:.4f}")
print(f"Optimal Lasso alpha      : {lasso_cv.alpha_:.4f}")
print(f"Optimal ElasticNet alpha : {elastic_cv.alpha_:.4f} (l1_ratio={elastic_cv.l1_ratio_})")
```

OUTPUT

```
Optimal Ridge alpha      : 19.1448
Optimal Lasso alpha      : 0.0100
Optimal ElasticNet alpha : 0.0257 (l1_ratio=0.2)
```

Step 7: Model Performance Benchmarking

CODE

```
models = {
    "OLS (LinearRegression)": ols,
    "Tuned Ridge": ridge_cv,
    "Tuned Lasso": lasso_cv,
    "Tuned ElasticNet": elastic_cv
}

eval_records = []
for name, m in models.items():
    preds = m.predict(X_test_scaled)
    mse = mean_squared_error(y_test, preds)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, preds)
    non_zeros = np.sum(np.abs(m.coef_) > 1e-4)
    eval_records.append({
        "Model": name,
        "Test RMSE": round(rmse, 4),
        "Test R2": round(r2, 4),
        "Active Features": non_zeros
    })

results_df = pd.DataFrame(eval_records)
print(results_df.to_string(index=False))
```

OUTPUT

```
                Model  Test RMSE  Test R2  Active Features
OLS (LinearRegression)     0.8774   0.4715               44
           Tuned Ridge     0.6558   0.7048               44
           Tuned Lasso     0.6679   0.6937               16
     Tuned ElasticNet     0.6633   0.6979               26
```

Experiment No. 6

Binary Classification using Logistic Regression: Confusion Matrix, Precision, Recall, and F1-Score

Dataset: Heart Disease (Statlog) Dataset (fetch_openml('heart-statlog', version=1))

Step 1: Import Required Libraries

CODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    confusion_matrix, classification_report, accuracy_score,
    roc_curve, auc, precision_recall_curve
)

sns.set_theme(style="whitegrid")
print("Libraries imported successfully.")
```

OUTPUT

```
Libraries imported successfully.
```

Step 2: Load Heart Disease Dataset

Heart Statlog contains 13 physiological features for 270 patients, predicting presence/absence of heart disease.

CODE

```
heart_data = fetch_openml('heart-statlog', version=1, as_frame=True)
df_heart = heart_data.frame.copy()

print(f"Dataset shape: {df_heart.shape}")
print(df_heart.head())

# Target: absent = 0, present = 1.
X = df_heart.drop(columns=['class']).to_numpy(dtype=float)
y = (df_heart['class'].astype(str) == 'present').astype(int).to_numpy()

print(f"Total samples: {len(y)}")
print(f"Negative (Absence - 0): {np.sum(y == 0)}")
print(f"Positive (Presence - 1): {np.sum(y == 1)}")
```

OUTPUT

```
Dataset shape: (270, 14)
   age  sex  chest  resting_blood_pressure  serum_cholestoral  \
0   70    1      4                     130                322
1   67    0      3                     115                564
2   57    1      2                     124                261
3   64    1      4                     128                263
4   74    0      2                     120                269

   fasting_blood_sugar  resting_electrocardiographic_results  \
0                    0                                    2
1                    0                                    2
2                    0                                    0
3                    0                                    0
4                    0                                    2

   maximum_heart_rate_achieved  exercise_induced_angina  oldpeak  slope  \
0                          109                        0      2.4      2
1                          160                        0      1.6      2
2                          141                        0      0.3      1
3                          105                        1      0.2      2
4                          121                        1      0.2      1

   number_of_major_vessels  thal    class
0                        3     3  present
```

```
1                       0     7   absent
2                       0     7  present
3                       1     7   absent
4                       1     3   absent
Total samples: 270
Negative (Absence - 0): 150
Positive (Presence - 1): 120
```

Step 3: Train-Test Split and Standardization

CODE

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"Training set: {X_train_scaled.shape} | Testing set: {X_test_scaled.shape}")
```

OUTPUT

```
Training set: (202, 13) | Testing set: (68, 13)
```

Step 4: Train Logistic Regression Classifier

CODE

```
log_reg = LogisticRegression(max_iter=1000, random_state=42)
log_reg.fit(X_train_scaled, y_train)

# Predict probabilities and discrete classes
y_pred = log_reg.predict(X_test_scaled)
y_prob = log_reg.predict_proba(X_test_scaled)[:, 1]

print("Model converged successfully.")
```

OUTPUT

```
Model converged successfully.
```

Step 5: Confusion Matrix & Classification Report

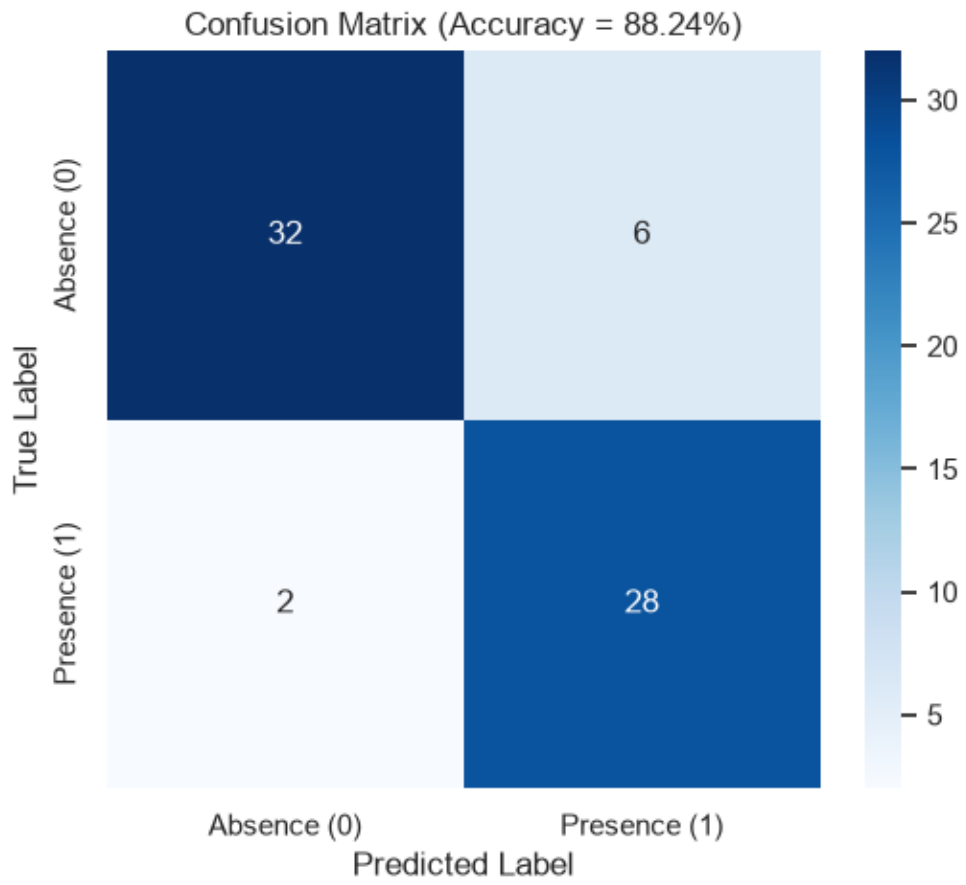
CODE

```
cm = confusion_matrix(y_test, y_pred)
acc = accuracy_score(y_test, y_pred)

plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Absence (0)', 'Presence (1)'],
            yticklabels=['Absence (0)', 'Presence (1)'])
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title(f"Confusion Matrix (Accuracy = {acc*100:.2f}%)")
plt.show()

print("Comprehensive Classification Report:")
print(classification_report(y_test, y_pred, target_names=['Absence', 'Presence']))
```

OUTPUT



OUTPUT

```
Comprehensive Classification Report:
              precision    recall  f1-score   support

     Absence       0.94      0.84      0.89        38
    Presence       0.82      0.93      0.88        30

    accuracy                           0.88        68
   macro avg       0.88      0.89      0.88        68
weighted avg       0.89      0.88      0.88        68
```

Step 6: Receiver Operating Characteristic (ROC) & Precision-Recall Curves

CODE

```
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

precision, recall, pr_thresholds = precision_recall_curve(y_test, y_prob)

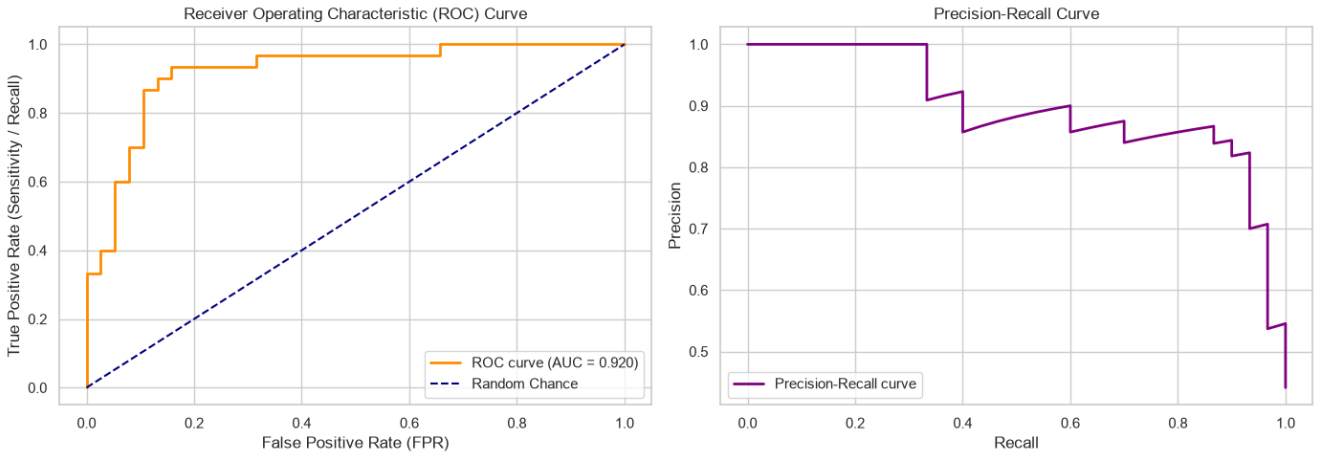
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# ROC Curve
axes[0].plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC = {roc_auc:.3f})')
axes[0].plot([0, 1], [0, 1], color='navy', lw=1.5, linestyle='--', label='Random Chance')
axes[0].set_xlabel("False Positive Rate (FPR)")
axes[0].set_ylabel("True Positive Rate (Sensitivity / Recall)")
axes[0].set_title("Receiver Operating Characteristic (ROC) Curve")
axes[0].legend(loc="lower right")

# Precision-Recall Curve
axes[1].plot(recall, precision, color='purple', lw=2, label='Precision-Recall curve')
axes[1].set_xlabel("Recall")
axes[1].set_ylabel("Precision")
axes[1].set_title("Precision-Recall Curve")
axes[1].legend(loc="lower left")

plt.tight_layout()
plt.show()
```

OUTPUT



Step 7: Logistic Regression Feature Importance (Odds Ratios)

CODE

```
feature_names = df_heart.drop(columns=['class']).columns
odds_ratios = np.exp(log_reg.coef_[0])

coef_df = pd.DataFrame({
    'Feature': feature_names,
    'Log-Odds (Weight)': log_reg.coef_[0],
    'Odds Ratio': odds_ratios
}).sort_values(by='Odds Ratio', ascending=False)

print("Feature Importance via Odds Ratios:")
print(coef_df.to_string(index=False))
```

OUTPUT

```
Feature Importance via Odds Ratios:
                          Feature  Log-Odds (Weight)  Odds Ratio
            number_of_major_vessels           1.112131    3.040833
                                sex           0.909800    2.483826
                              chest           0.700321    2.014400
                               thal           0.522901    1.686913
                              slope           0.462438    1.587941
            exercise_induced_angina           0.450357    1.568872
             resting_blood_pressure           0.363737    1.438696
                  serum_cholestoral           0.331812    1.393491
                            oldpeak           0.265453    1.304022
resting_electrocardiographic_results           0.037412    1.038121
                                age          -0.004825    0.995186
                fasting_blood_sugar          -0.348902    0.705462
         maximum_heart_rate_achieved          -0.396052    0.672972
```

Experiment No. 7

Support Vector Machine Classification: Linear and Non-linear Kernels

Dataset: Banknote Authentication (fetch_openml('banknote-authentication', version=1))

Step 1: Import Required Libraries

CODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.decomposition import PCA
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix

sns.set_theme(style="whitegrid")
print("Libraries imported successfully.")
```

OUTPUT

```
Libraries imported successfully.
```

Step 2: Load and Prepare Banknote Dataset

Extracted from continuous wavelet transform features of genuine vs counterfeit banknote images.

CODE

```
banknote_data = fetch_openml('banknote-authentication', version=1, as_frame=True)
df_bn = banknote_data.frame.copy()

print(f"Dataset shape: {df_bn.shape}")
print(df_bn.head())

X = df_bn[['V1', 'V2', 'V3', 'V4']].to_numpy(dtype=float)
y = (df_bn['Class'].astype(str) == '2').astype(int).to_numpy()

print(f"Class Distribution: {np.bincount(y)} (0: Genuine, 1: Counterfeit)")
```

OUTPUT

```
Dataset shape: (1372, 5)
        V1      V2      V3       V4 Class
0  3.62160  8.6661 -2.8073 -0.44699     1
1  4.54590  8.1674 -2.4586 -1.46210     1
2  3.86600 -2.6383  1.9242  0.10645     1
3  3.45660  9.5228 -4.0112 -3.59440     1
4  0.32924 -4.4552  4.5718 -0.98880     1
Class Distribution: [762 610] (0: Genuine, 1: Counterfeit)
```

Step 3: Train-Test Split and Standardization

CODE

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Step 4: Comparing SVM Kernels (Linear, Polynomial, RBF, Sigmoid)

CODE

```
kernels = ['linear', 'poly', 'rbf', 'sigmoid']
kernel_results = []

for k in kernels:
    svc = SVC(kernel=k, C=1.0, random_state=42)
    svc.fit(X_train_scaled, y_train)
    y_pred = svc.predict(X_test_scaled)
    acc = accuracy_score(y_test, y_pred)
```

```
    n_sv = svc.n_support_.sum()
    kernel_results.append({
        'Kernel': k.upper(),
        'Accuracy (%)': round(acc * 100, 2),
        'Total Support Vectors': n_sv,
        'Support Vectors per Class': list(svc.n_support_)
    })

results_df = pd.DataFrame(kernel_results)
print("Kernel Comparison Benchmarking Table:")
print(results_df.to_string(index=False))
```

OUTPUT

```
Kernel Comparison Benchmarking Table:
 Kernel  Accuracy (%)  Total Support Vectors Support Vectors per Class
 LINEAR         98.06                     59                  [29, 30]
   POLY         98.79                    263                 [131, 132]
    RBF        100.00                     88                  [42, 46]
SIGMOID         78.16                    247                 [124, 123]
```

Step 5: Hyperparameter Optimization Grid Search (C and Gamma)

CODE

```
param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': ['scale', 0.01, 0.1, 1],
    'kernel': ['rbf']
}

grid = GridSearchCV(SVC(random_state=42), param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid.fit(X_train_scaled, y_train)

best_svc = grid.best_estimator_
print(f"Optimal Hyperparameters: {grid.best_params_}")
print(f"Best Cross-Validation Accuracy: {grid.best_score_*100:.2f}%")

test_acc = accuracy_score(y_test, best_svc.predict(X_test_scaled))
print(f"Test Set Accuracy: {test_acc*100:.2f}%")
```

OUTPUT

```
Optimal Hyperparameters: {'C': 10, 'gamma': 'scale', 'kernel': 'rbf'}
Best Cross-Validation Accuracy: 100.00%
Test Set Accuracy: 100.00%
```

Step 6: 2D Decision Boundary Visualization

We project the 4D feature space onto 2 principal components to visualize the non-linear RBF decision boundary.

CODE

```
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train_scaled)

# Fit SVM on 2D PCA representation for visualization
C_opt = grid.best_params_['C']
gamma_opt = grid.best_params_['gamma']
svm_2d = SVC(kernel='rbf', C=C_opt, gamma=gamma_opt)
svm_2d.fit(X_train_pca, y_train)

# Create dense 2D meshgrid
x_min, x_max = X_train_pca[:, 0].min() - 0.8, X_train_pca[:, 0].max() + 0.8
y_min, y_max = X_train_pca[:, 1].min() - 0.8, X_train_pca[:, 1].max() + 0.8
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 400), np.linspace(y_min, y_max, 400))

# Decision function (distance to separating hyperplane)
Z = svm_2d.decision_function(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

plt.figure(figsize=(10, 7), dpi=120)

# 1. Soft shaded decision regions
plt.contourf(xx, yy, Z > 0, alpha=0.18, cmap='coolwarm')

# 2. Precise Decision Boundary (f(x)=0) and Margins (f(x)= +/- 1)
contours = plt.contour(
    xx, yy, Z,
    levels=[-1.0, 0.0, 1.0],
    colors=['#444444', '#111111', '#444444'],
    linestyles=['--', '-', '--'],
```

```
    linewidths=[1.3, 2.2, 1.3]
)
plt.clabel(contours, fmt={-1.0: 'Margin (-1)', 0.0: 'Boundary (0)', 1.0: 'Margin (+1)'}, fontsize=9, inl
    ine=True)

# 3. Scatter data points by class
scatter_0 = plt.scatter(
    X_train_pca[y_train == 0, 0], X_train_pca[y_train == 0, 1],
    color='#3b6cb7', edgecolor='white', linewidth=0.5, s=35, alpha=0.85, label='Class 0: Genuine'
)
scatter_1 = plt.scatter(
    X_train_pca[y_train == 1, 0], X_train_pca[y_train == 1, 1],
    color='#c93b3b', edgecolor='white', linewidth=0.5, s=35, alpha=0.85, label='Class 1: Counterfeit'
)

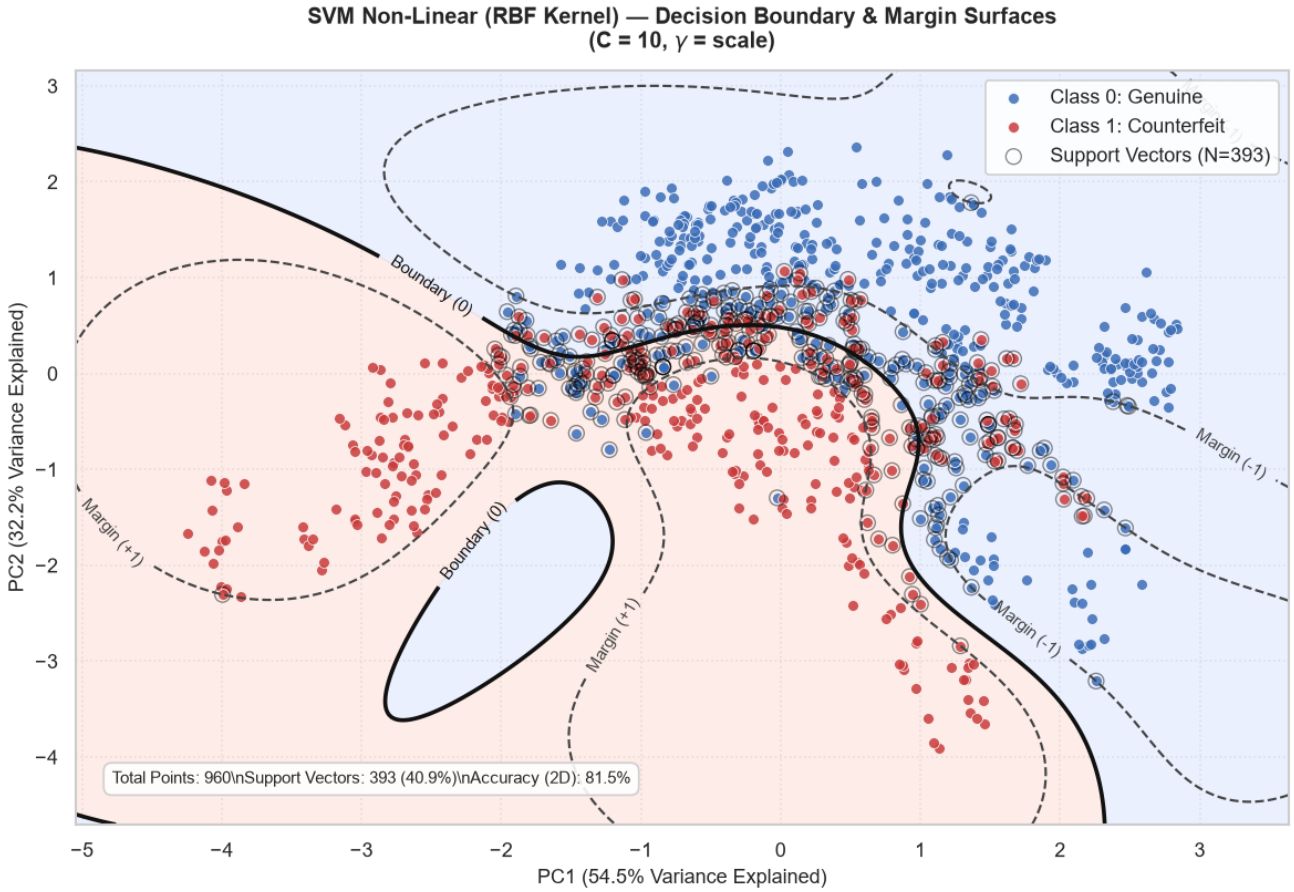
# 4. Highlight Support Vectors cleanly with delicate rings
sv = svm_2d.support_vectors_
plt.scatter(
    sv[:, 0], sv[:, 1],
    s=70, facecolors='none', edgecolors='black', linewidths=0.8, alpha=0.45,
    label=f'Support Vectors (N={len(sv)})'
)

# Annotations & styling
plt.title(f"SVM Non-Linear (RBF Kernel) - Decision Boundary & Margin Surfaces\n(C = {C_opt}, $\\gamma$ =
     {gamma_opt})", fontsize=12, pad=12, fontweight='bold')
plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0]*100:.1f}% Variance Explained)", fontsize=11)
plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1]*100:.1f}% Variance Explained)", fontsize=11)

# Informative stats box
stats_text = f"Total Points: {len(X_train)}\\nSupport Vectors: {len(sv)} ({len(sv)/len(X_train)*100:.1f}
    %)\\nAccuracy (2D): {svm_2d.score(X_train_pca, y_train)*100:.1f}%"
plt.gca().text(
    0.03, 0.05, stats_text, transform=plt.gca().transAxes,
    fontsize=9, verticalalignment='bottom',
    bbox=dict(boxstyle='round,pad=0.5', facecolor='white', edgecolor='#cccccc', alpha=0.9)
)

plt.legend(loc='upper right', frameon=True, facecolor='white', framealpha=0.9)
plt.grid(True, linestyle=':', alpha=0.4)
plt.tight_layout()
plt.show()
```

OUTPUT


SVM Non-Linear (RBF Kernel) — Decision Boundary & Margin Surfaces
(C = 10, γ = scale)
Class 0: Genuine
Class 1: Counterfeit
Support Vectors (N=393)
Boundary (0)
Boundary (0)
Margin (+1)
Margin (+1)
Margin (-1)
Margin (-1)
Total Points: 960\nSupport Vectors: 393 (40.9%)\nAccuracy (2D): 81.5%
PC2 (32.2% Variance Explained)
PC1 (54.5% Variance Explained)
3
2
1
0
−1
−2
−3
−4
−5
−4
−3
−2
−1
0
1
2
3

Experiment No. 8

K-Means Clustering with Optimal Cluster Selection using Elbow / Silhouette Method

Dataset: Wheat Seeds Dataset (fetch_openml('seeds', version=1))

Step 1: Import Required Libraries

CODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_openml
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score, silhouette_samples, calinski_harabasz_score, davies_bouldi
    n_score

sns.set_theme(style="whitegrid")
print("Libraries imported successfully.")
```

OUTPUT

```
Libraries imported successfully.
```

Step 2: Load and Prepare Wheat Seeds Dataset

Contains 7 geometric features of wheat kernels across 3 distinct varieties (Kama, Rosa, Canadian).

CODE

```
seeds_data = fetch_openml('seeds', version=1, as_frame=True)
df_seeds = seeds_data.frame.copy()

print(f"Dataset shape: {df_seeds.shape}")
print(df_seeds.head())

feature_cols = ['V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7']
X = df_seeds[feature_cols].to_numpy(dtype=float)
y_true = df_seeds['Class'].astype(int).to_numpy()

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

OUTPUT

```
Dataset shape: (210, 8)
      V1     V2      V3     V4     V5     V6     V7 Class
0  15.26  14.84  0.8710  5.763  3.312  2.221  5.220     1
1  14.88  14.57  0.8811  5.554  3.333  1.018  4.956     1
2  14.29  14.09  0.9050  5.291  3.337  2.699  4.825     1
3  13.84  13.94  0.8955  5.324  3.379  2.259  4.805     1
4  16.14  14.99  0.9034  5.658  3.562  1.355  5.175     1
```

Step 3: Within-Cluster Sum of Squares (WCSS) - Elbow Method

We compute inertia across k in 1 ... 10 to find the inflection 'elbow' point.

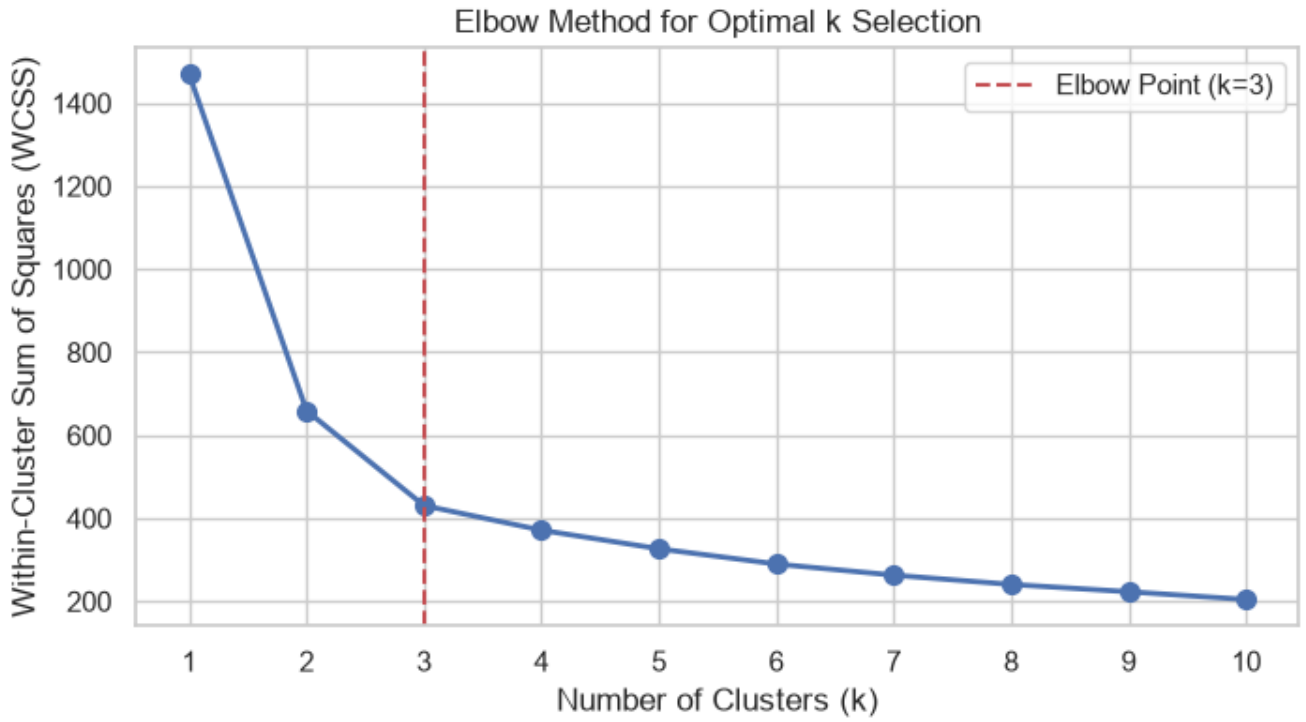
CODE

```
k_range = range(1, 11)
wcss = []

for k in k_range:
    kmeans = KMeans(n_clusters=k, init='k-means++', random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

plt.figure(figsize=(8, 4))
plt.plot(k_range, wcss, 'bo-', linewidth=2, markersize=7)
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Within-Cluster Sum of Squares (WCSS)")
plt.title("Elbow Method for Optimal k Selection")
plt.xticks(k_range)
plt.axvline(3, color='r', linestyle='--', label='Elbow Point (k=3)')
plt.legend()
plt.show()
```

OUTPUT



Step 4: Silhouette Score Profiling

We compute the mean silhouette coefficient across k in 2 ... 10 to quantify cluster cohesion and separation.

CODE

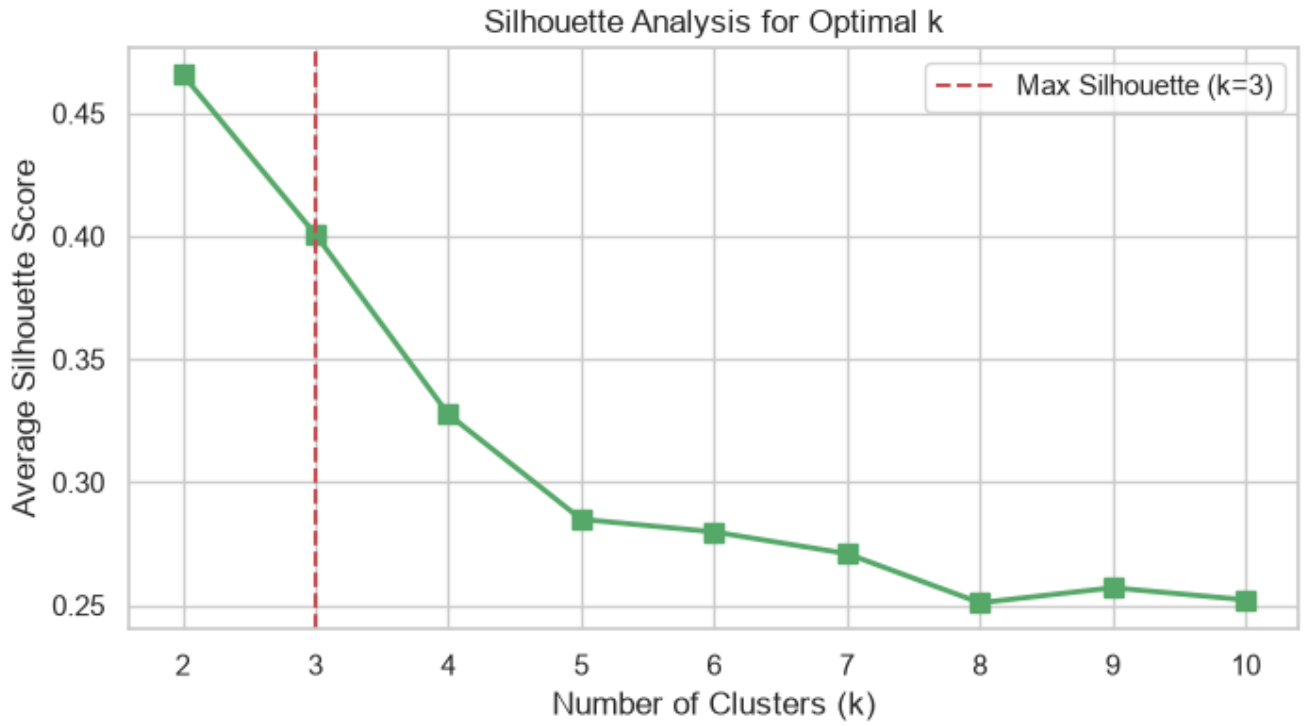
```
silhouette_scores = []

for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, init='k-means++', random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)
    silhouette_scores.append(score)

plt.figure(figsize=(8, 4))
plt.plot(range(2, 11), silhouette_scores, 'gs-', linewidth=2, markersize=7)
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Average Silhouette Score")
plt.title("Silhouette Analysis for Optimal k")
plt.xticks(range(2, 11))
plt.axvline(3, color='r', linestyle='--', label='Max Silhouette (k=3)')
plt.legend()
plt.show()

for k, score in zip(range(2, 11), silhouette_scores):
    print(f"k = {k:2d} | Silhouette Score: {score:.4f}")
```

OUTPUT



OUTPUT

```
k =  2 | Silhouette Score: 0.4658
k =  3 | Silhouette Score: 0.4007
k =  4 | Silhouette Score: 0.3276
k =  5 | Silhouette Score: 0.2849
k =  6 | Silhouette Score: 0.2798
k =  7 | Silhouette Score: 0.2709
k =  8 | Silhouette Score: 0.2510
k =  9 | Silhouette Score: 0.2571
k = 10 | Silhouette Score: 0.2522
```

Step 5: Fit Optimal K-Means Model (k=3) & Evaluate Quality Metrics

CODE

```
optimal_k = 3
kmeans_opt = KMeans(n_clusters=optimal_k, init='k-means++', random_state=42, n_init=10)
cluster_labels = kmeans_opt.fit_predict(X_scaled)

sil = silhouette_score(X_scaled, cluster_labels)
ch = calinski_harabasz_score(X_scaled, cluster_labels)
db = davies_bouldin_score(X_scaled, cluster_labels)

metrics_summary = pd.DataFrame({
    'Metric': ['Silhouette Score (Higher is better)', 'Calinski-Harabasz Index (Higher is better)', 'Dav
    ies-Bouldin Index (Lower is better)'],
    'Value': [round(sil, 4), round(ch, 2), round(db, 4)]
})
print("Clustering Validation Metrics Summary:")
print(metrics_summary.to_string(index=False))
```

OUTPUT

```
Clustering Validation Metrics Summary:
                                Metric    Value
       Silhouette Score (Higher is better)   0.4007
Calinski-Harabasz Index (Higher is better) 249.7800
    Davies-Bouldin Index (Lower is better)   0.9279
```

Step 6: 2D PCA Visualization with Cluster Centroids

CODE

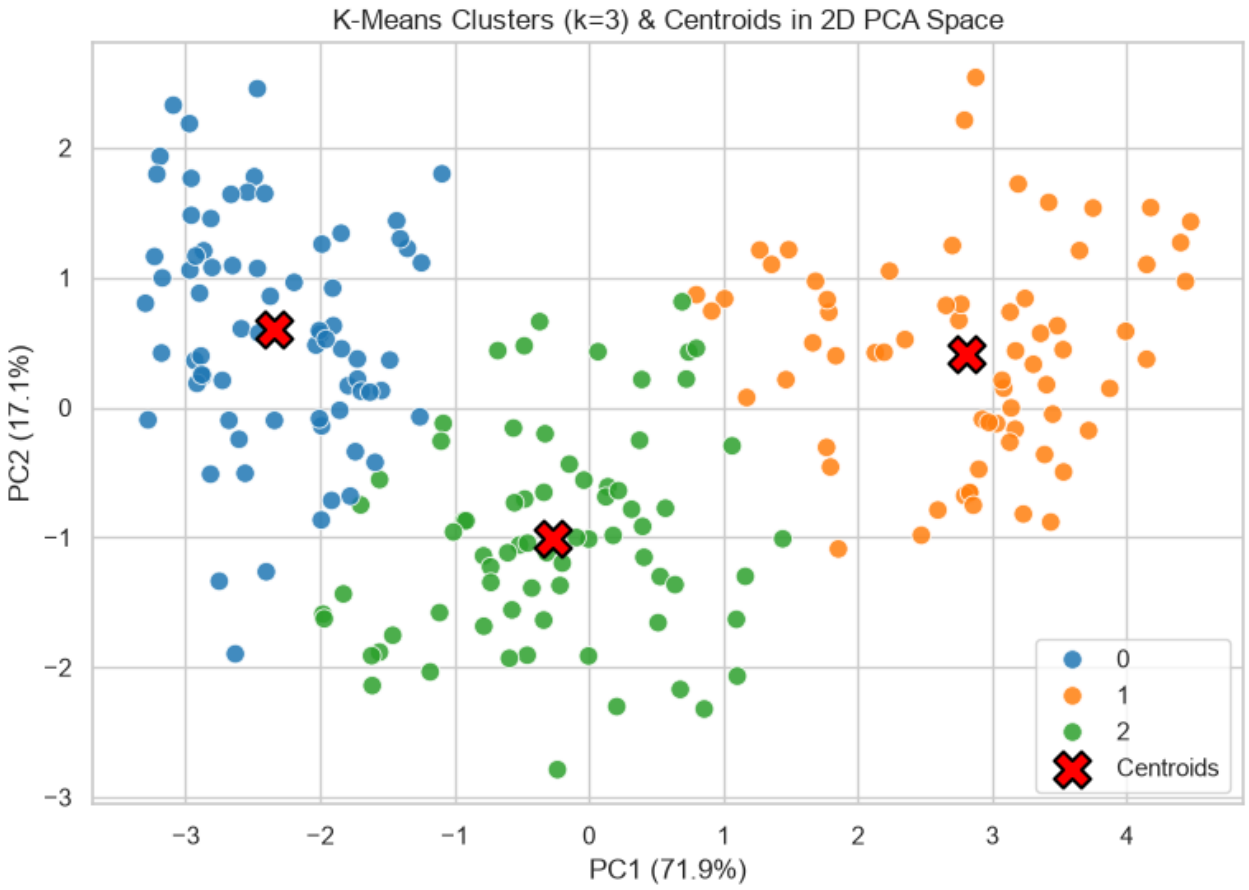
```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
centroids_pca = pca.transform(kmeans_opt.cluster_centers_)

plt.figure(figsize=(9, 6))
sns.scatterplot(x=X_pca[:, 0], y=X_pca[:, 1], hue=cluster_labels, palette='tab10', s=70, alpha=0.85)
```

```
plt.scatter(centroids_pca[:, 0], centroids_pca[:, 1], s=250, c='red', marker='X', edgecolor='black', lin
    ewidth=1.5, label='Centroids')

plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0]*100:.1f}%)")
plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1]*100:.1f}%)")
plt.title("K-Means Clusters (k=3) & Centroids in 2D PCA Space")
plt.legend()
plt.show()
```

OUTPUT



Experiment No. 9

Density-Based Clustering using DBSCAN

Dataset: Concentric Density Rings with Outlier Noise (sklearn.datasets.make_circles + uniform noise)

Step 1: Import Required Libraries

CODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import make_circles
from sklearn.cluster import DBSCAN, KMeans
from sklearn.neighbors import NearestNeighbors
from sklearn.metrics import silhouette_score

sns.set_theme(style="whitegrid")
print("Libraries imported successfully.")
```

OUTPUT

```
Libraries imported successfully.
```

Step 2: Generate Non-Convex Spatial Dataset with Background Noise

We construct concentric circular manifolds embedded with random spatial noise points.

CODE

```
np.random.seed(42)

# Generate inner and outer circles
X_circles, _ = make_circles(n_samples=500, factor=0.4, noise=0.05, random_state=42)

# Add uniform random background noise
X_noise = np.random.uniform(low=-1.3, high=1.3, size=(50, 2))

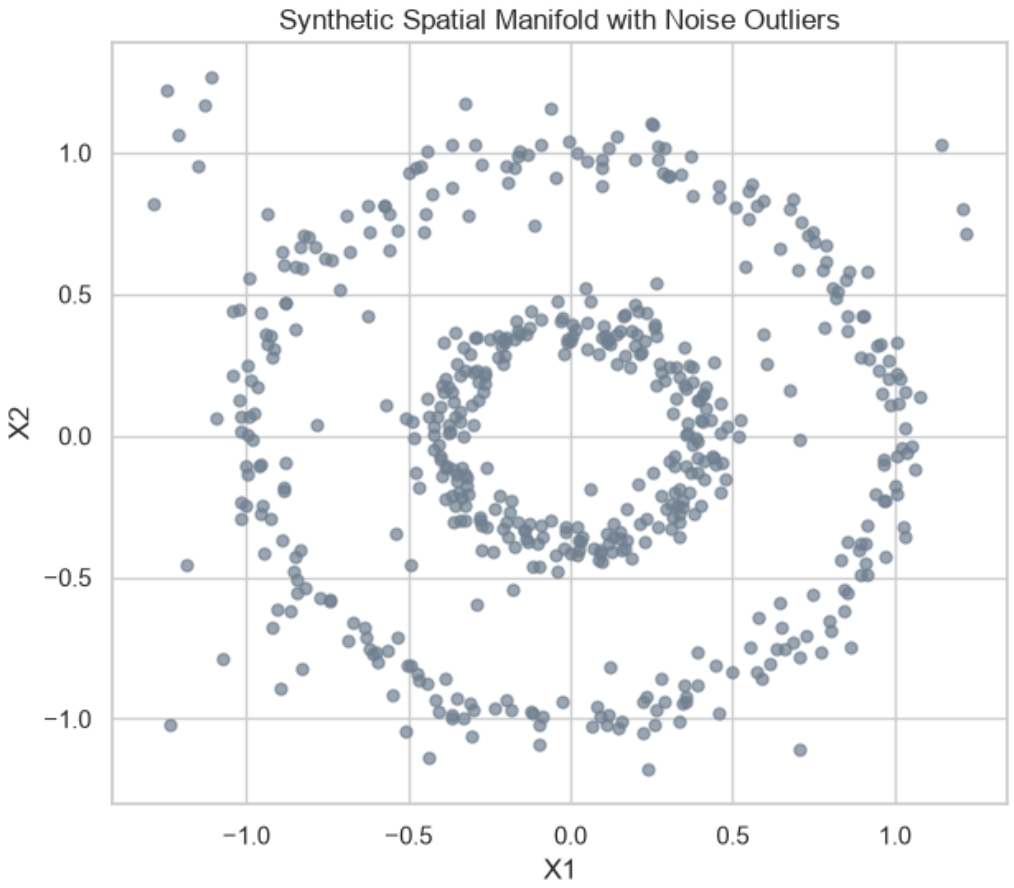
# Combine datasets
X = np.vstack([X_circles, X_noise])
print(f"Total dataset shape: {X.shape} (500 cluster points + 50 noise outliers)")

plt.figure(figsize=(7, 6))
plt.scatter(X[:, 0], X[:, 1], color='slategray', s=25, alpha=0.7)
plt.title("Synthetic Spatial Manifold with Noise Outliers")
plt.xlabel("X1")
plt.ylabel("X2")
plt.show()
```

OUTPUT

```
Total dataset shape: (550, 2) (500 cluster points + 50 noise outliers)
```

OUTPUT



Step 3: Determine Optimal Epsilon (eps) via Sorted k-Distance Graph

By plotting sorted distances to the k-th nearest neighbor (k = {min_samples}), the sharp 'knee' indicates optimal eps.

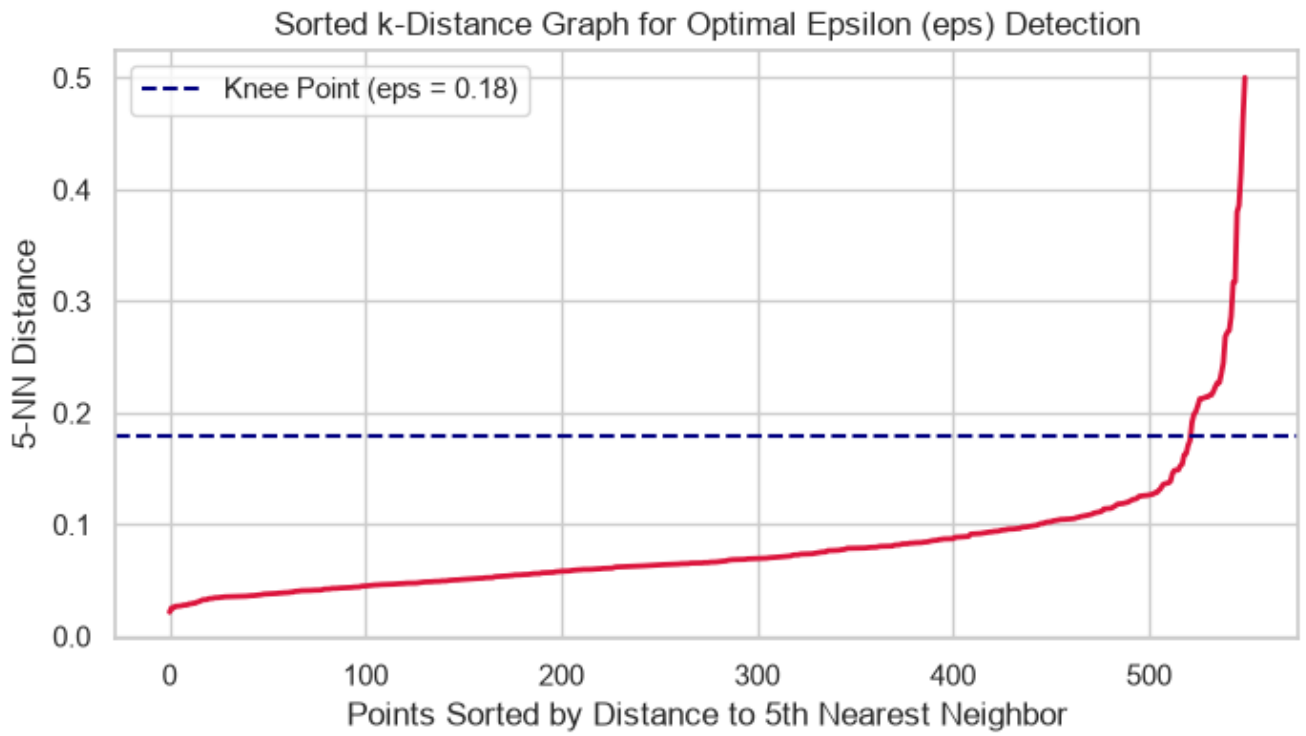
CODE

```
k = 5
nn = NearestNeighbors(n_neighbors=k)
nn.fit(X)
distances, indices = nn.kneighbors(X)

# Sort distances to the 5th neighbor
k_distances = np.sort(distances[:, k - 1])

plt.figure(figsize=(8, 4))
plt.plot(k_distances, color='crimson', lw=2)
plt.axhline(y=0.18, color='navy', linestyle='--', label='Knee Point (eps = 0.18)')
plt.xlabel("Points Sorted by Distance to 5th Nearest Neighbor")
plt.ylabel("5-NN Distance")
plt.title("Sorted k-Distance Graph for Optimal Epsilon (eps) Detection")
plt.legend()
plt.show()
```

OUTPUT



Step 4: Fit DBSCAN & Identify Core, Border, and Noise Points

CODE

```
dbscan = DBSCAN(eps=0.18, min_samples=5)
labels = dbscan.fit_predict(X)

# Core points
core_mask = np.zeros_like(labels, dtype=bool)
core_mask[dbscan.core_sample_indices_] = True

n_clusters_ = len(set(labels)) - (1 if -1 in labels else 0)
n_noise_ = np.sum(labels == -1)

print(f"Number of discovered density clusters: {n_clusters_}")
print(f"Number of identified noise outliers : {n_noise_} ({n_noise_/len(X)*100:.1f}%)")
print(f"Number of core samples               : {core_mask.sum()}")
print(f"Number of border samples             : {len(X) - core_mask.sum() - n_noise_}")
```

OUTPUT

```
Number of discovered density clusters: 2
Number of identified noise outliers : 18 (3.3%)
Number of core samples               : 522
Number of border samples             : 10
```

Step 5: Side-by-Side Comparison: DBSCAN vs. K-Means

Demonstrates why distance-based partitioning (K-Means) fails on non-convex geometric manifolds.

CODE

```
kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)
km_labels = kmeans.fit_predict(X)

fig, axes = plt.subplots(1, 2, figsize=(15, 6))

# K-Means clustering
axes[0].scatter(X[:, 0], X[:, 1], c=km_labels, cmap='viridis', s=30, alpha=0.8)
axes[0].scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],
                s=200, c='red', marker='X', edgecolor='k', label='Centroids')
axes[0].set_title("K-Means (Failed on Non-Spherical Geometry)")
axes[0].set_xlabel("X1")
axes[0].set_ylabel("X2")
axes[0].legend()

# DBSCAN clustering
unique_labels = set(labels)
colors = [plt.cm.Spectral(each) for each in np.linspace(0, 1, len(unique_labels))]
```

```
for k_label, col in zip(unique_labels, colors):
    if k_label == -1:
        col = [0, 0, 0, 1]  # Black for noise
    class_member_mask = (labels == k_label)

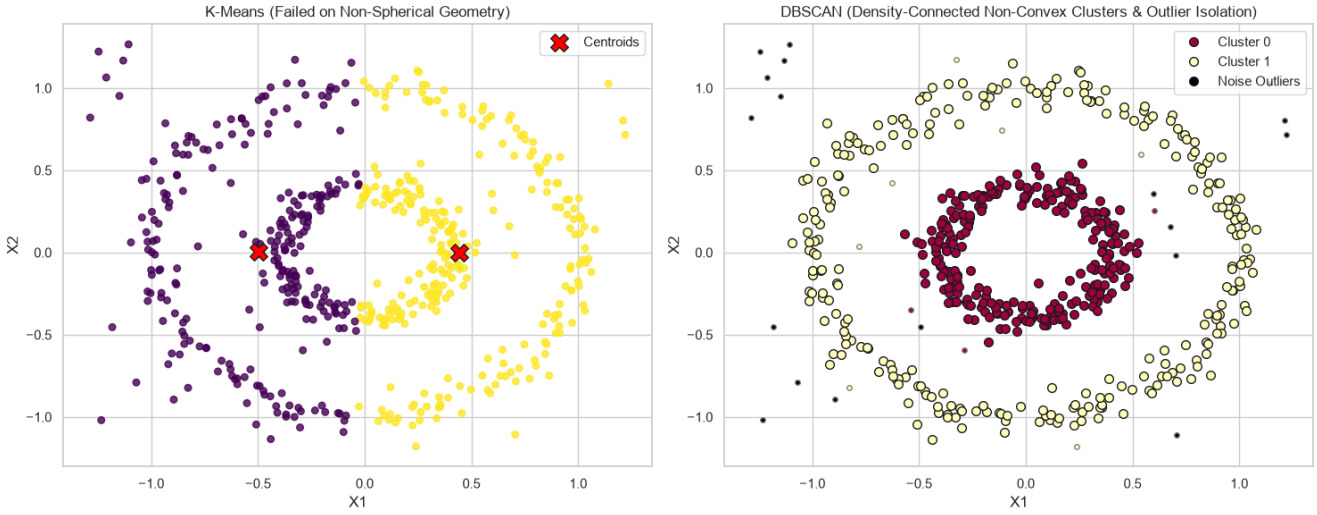
    xy = X[class_member_mask & core_mask]
    axes[1].plot(xy[:, 0], xy[:, 1], 'o', markerfacecolor=tuple(col),
                 markeredgecolor='k', markersize=7, label=f'Cluster {k_label}' if k_label != -1 else 'No
    ise Outliers')

    xy_noise = X[class_member_mask & ~core_mask]
    axes[1].plot(xy_noise[:, 0], xy_noise[:, 1], 'o', markerfacecolor=tuple(col),
                 markeredgecolor='gray', markersize=4)

axes[1].set_title("DBSCAN (Density-Connected Non-Convex Clusters & Outlier Isolation)")
axes[1].set_xlabel("X1")
axes[1].set_ylabel("X2")
axes[1].legend(loc='upper right')

plt.tight_layout()
plt.show()
```

OUTPUT



Step 6: Parameter Sensitivity Grid

CODE

```
eps_vals = [0.10, 0.18, 0.25]
min_samples_vals = [3, 5, 10]

grid_records = []
for e in eps_vals:
    for ms in min_samples_vals:
        db = DBSCAN(eps=e, min_samples=ms)
        lbs = db.fit_predict(X)
        n_c = len(set(lbs)) - (1 if -1 in lbs else 0)
        n_n = np.sum(lbs == -1)
        grid_records.append({
            'eps': e,
            'min_samples': ms,
            'Discovered Clusters': n_c,
            'Noise Outliers': n_n
        })

print(pd.DataFrame(grid_records).to_string(index=False))
```

OUTPUT

```
 eps  min_samples  Discovered Clusters  Noise Outliers
0.10            3                    5              39
0.10            5                   11              44
0.10           10                    5             252
0.18            3                    3               8
0.18            5                    2              18
0.18           10                    2              18
0.25            3                    3               2
0.25            5                    2               6
```

```
0.25            10                  1               11
```

Experiment No. 10

Ensemble Learning: Random Forest / Boosting vs Baseline Model

Dataset: Ionosphere Radar Dataset (fetch_openml('ionosphere', version=1))

Step 1: Import Required Libraries

CODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (
    RandomForestClassifier, AdaBoostClassifier, GradientBoostingClassifier,
    VotingClassifier, StackingClassifier
)
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score

sns.set_theme(style="whitegrid")
print("Libraries imported successfully.")
```

OUTPUT

```
Libraries imported successfully.
```

Step 2: Load and Prepare Ionosphere Radar Dataset

Contains 34 continuous pulse signals collected by NASA radar system predicting whether radar returns are 'Good' (structure in ionosphere) or 'Bad'.

CODE

```
iono_data = fetch_openml('ionosphere', version=1, as_frame=True)
df_iono = iono_data.frame.copy()

print(f"Dataset shape: {df_iono.shape}")
print(df_iono.head())

# Target: 'g' = 1, 'b' = 0
y = (df_iono['class'].astype(str) == 'g').astype(int).to_numpy()
X = df_iono.drop(columns=['class']).to_numpy(dtype=float)

# Drop any constant columns
variance = np.var(X, axis=0)
valid_cols = variance > 1e-6
X = X[:, valid_cols]

print(f"Remaining active feature columns: {X.shape[1]}")
print(f"Target distribution: {np.bincount(y)} (0: Bad, 1: Good)")
```

OUTPUT

```
Dataset shape: (351, 35)
   a01  a02      a03      a04      a05      a06      a07      a08      a09  \
0    1    0  0.99539 -0.05889  0.85243  0.02306  0.83398 -0.37708  1.00000
1    1    0  1.00000 -0.18829  0.93035 -0.36156 -0.10868 -0.93597  1.00000
2    1    0  1.00000 -0.03365  1.00000  0.00485  1.00000 -0.12062  0.88965
3    1    0  1.00000 -0.45161  1.00000  1.00000  0.71216 -1.00000  0.00000
4    1    0  1.00000 -0.02401  0.94140  0.06531  0.92106 -0.23255  0.77152

       a10  ...      a26      a27      a28      a29      a30      a31  \
0  0.03760  ... -0.51171  0.41078 -0.46168  0.21266 -0.34090  0.42267
1 -0.04549  ... -0.26569 -0.20468 -0.18401 -0.19040 -0.11593 -0.16626
2  0.01198  ... -0.40220  0.58984 -0.22145  0.43100 -0.17365  0.60436
3  0.00000  ...  0.90695  0.51613  1.00000  1.00000 -0.20099  0.25682
4 -0.16399  ... -0.65158  0.13290 -0.53206  0.02431 -0.62197 -0.05707

       a32      a33      a34  class
0 -0.54487  0.18641 -0.45300      g
1 -0.06288 -0.13738 -0.02447      b
2 -0.24180  0.56045 -0.38238      g
```

```
3  1.00000 -0.32382  1.00000      b
4 -0.59573 -0.04608 -0.65697      g

[5 rows x 35 columns]
Remaining active feature columns: 33
Target distribution: [126 225] (0: Bad, 1: Good)
```

Step 3: Train-Test Split and Feature Scaling

CODE

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Step 4: Define Baseline vs. Bagging, Boosting, and Stacking Models

CODE

```
# 1. Baseline Decision Tree
dt = DecisionTreeClassifier(random_state=42)

# 2. Bagging: Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42)

# 3. Boosting: AdaBoost
ada = AdaBoostClassifier(n_estimators=100, random_state=42)

# 4. Boosting: Gradient Boosting
gb = GradientBoostingClassifier(n_estimators=100, random_state=42)

# 5. Soft Voting Ensemble
voting = VotingClassifier(
    estimators=[('dt', dt), ('rf', rf), ('gb', gb)],
    voting='soft'
)

# 6. Stacking Classifier with Logistic Regression Meta-Learner
stacking = StackingClassifier(
    estimators=[('dt', dt), ('rf', rf), ('gb', gb)],
    final_estimator=LogisticRegression(),
    cv=5
)

ensemble_models = {
    "Baseline: Decision Tree": dt,
    "Bagging: Random Forest": rf,
    "Boosting: AdaBoost": ada,
    "Boosting: Gradient Boosting": gb,
    "Ensemble: Soft Voting": voting,
    "Ensemble: Stacking": stacking
}
```

Step 5: Benchmark and Evaluate Ensemble Architecture

CODE

```
results = []

for name, model in ensemble_models.items():
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    y_prob = model.predict_proba(X_test_scaled)[:, 1]

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    auc_score = roc_auc_score(y_test, y_prob)

    results.append({
        "Model Architecture": name,
        "Accuracy (%)": round(acc * 100, 2),
        "Precision": round(prec, 4),
        "Recall": round(rec, 4),
```

```
        "F1-Score": round(f1, 4),
        "ROC-AUC": round(auc_score, 4)
    })

comparison_df = pd.DataFrame(results)
print("Ensemble Learning vs. Baseline Benchmark:")
print(comparison_df.to_string(index=False))
```

OUTPUT

```
Ensemble Learning vs. Baseline Benchmark:
         Model Architecture  Accuracy (%)  Precision  Recall  F1-Score  ROC-AUC
    Baseline: Decision Tree         86.36     0.8548  0.9464    0.8983   0.8326
     Bagging: Random Forest         93.18     0.9310  0.9643    0.9474   0.9501
         Boosting: AdaBoost         93.18     0.9032  1.0000    0.9492   0.9517
Boosting: Gradient Boosting         93.18     0.9167  0.9821    0.9483   0.9475
      Ensemble: Soft Voting         92.05     0.9153  0.9643    0.9391   0.9506
         Ensemble: Stacking         94.32     0.9322  0.9821    0.9565   0.9528
```

Step 6: Feature Importance Analysis (Random Forest)

CODE

```
importances = rf.feature_importances_
top_indices = np.argsort(importances)[::-1][:10]

plt.figure(figsize=(10, 5))
plt.bar(range(10), importances[top_indices], color='indigo', alpha=0.85)
plt.xticks(range(10), [f"Feat {i}" for i in top_indices])
plt.xlabel("Top 10 Informative Features")
plt.ylabel("Gini Feature Importance")
plt.title("Random Forest Feature Importance Profile")
plt.show()
```

OUTPUT

